

REN-203: Messaging

What's in a Message?

At the core, Rhapsody receives discrete blocks of data through the communication points. In most cases, the blocks are pre-defined (for example, as a file on disk), and in other cases the blocks are defined by agreed markers in the data stream (for example, input via the TCP communication points).

Each of these discrete data blocks forms a message to be handled by Rhapsody. Rhapsody's view of the message is as a complex object, consisting of:

- The message body: the content of the message as received on a communication point.
- Properties: meta-data extracted during processing by the route pipeline, either explicitly or implicitly.
- Other meta-data: other data about the message derived by the engine and maintained with the message object.

The message object is saved to the Datastore whenever message properties or the body change. The Datastore handles the message properties and the body separately for efficiency. Each message is saved a minimum of three times:

1. When the message is received by the Input communication point.
2. When the message passes to a route queue.
3. When the message passes from the route queue to an Output communication point

The content of the message (the message body) is generally carried in (human) readable text, although Rhapsody handles binary messages equally well. When Rhapsody receives a message by Rhapsody, the engine will attempt to determine how the content is encoded to ensure it manages the message and extracts values correctly. Where the encoding is known, it may be defined in the communication point properties to ensure the correct encoding is used.

Overview

The HL7 Message Modifier is a Rhapsody filter that operates on HL7 messages using an inbuilt generic message definition. It is designed to quickly and easily perform most commonly-used modifications on HL7 messages being processed by Rhapsody. It is the best choice for manipulating messages when only small changes are required.

Character Encoding

Users normally think messages are readable without any additional processing. The message is actually a stream of bytes with an associated set of rules which define how those bytes are interpreted and how they are to be presented for display. These rules are contained in the definition of the character encoding scheme used for the message and link the value, the meaning, and the glyph used to display the value.

Simple encoding schemes

Users in the English-speaking world are familiar with a simple character set comprising upper and lower case alphabetic characters (A-Z and a-z), the number set (0-9) and the set of punctuation characters. Early in the development of information technology, the ASCII character encoding (American Standard Code for Information Interchange) was defined to handle this character set. The ASCII code set was handled in 7 bits and supports:

- A range of special and non-printable control characters in values 0 to 31. The value 0 has a special meaning in many programming languages and may be used to terminate a character string. It is therefore not used for other purposes.
- Space, punctuation, and numbers in values 32 to 64.
- Upper case alphabetic characters and punctuation characters in values 65 to 96.
- Lower case alphabetic characters and additional punctuation characters in values 97 to 127.

A number of additional schemes were implemented by a range of vendors to use the 8th bit, encoding values between 128 and 255. The most commonly known is the Windows encoding scheme (known as Windows-1252 or Latin and including a range of line drawing codes and some special characters).

The simple encoding model fails to support more complex character sets. Consequently, a range of alternate encoding models were derived to provide greater flexibility. We will examine the Unicode (UTF) models as the most frequently used.

Unicode Character Sets

Many languages have character sets which exceed the ability of the simple encoding schemes. For example, some Asian languages contain many thousands of characters while other language sets also present characters right to left instead of left to right, which impacts both presentation and interpretation.

The Unicode standard was established to provide the flexibility to handle extended character sets.

Unicode

Unicode defines a Unicode code point to represent a character value. Due to the number of possible code points, a flexible scheme is required to manage the code points; the most common of these are UTF8 and UTF16. They differ primarily in the way the values are encoded:

- UTF8
The code points are represented in 1 or more 8 bit values,
- UTF16
The code points are represented in 1 or more 16 bit values.

Code point values may be referenced with the general notation U+xxxx where xxxx is a hexadecimal number; for example, H is U+0048, while h is U+0068 (note that Rhapsody uses the Java notation \uxxxx). The first 127 code points parallel the ASCII character set in all encodings.

High code point values are managed by using continuation markers at the start of each value; for example, UTF8 uses a marker in the first byte to indicate how many bytes are needed to represent the code point, and a continuation marker on each subsequent byte.

While there is generally nothing intrinsic in the data stream to indicate the encoding system, there may optionally be a marker called the Byte Order Mark (BOM) at the beginning of the data. As an example, if a UTF8 file is created or edited by the Windows Notepad editor, the BOM will be saved with the file.

Surrogate characters

The Unicode specification normally utilizes code points to map to locations in a single address space (known as the Basic Multilingual Plane or BMP). A number of additional address spaces (Planes) are available for use if required.

Some encoding schemes utilizing UTF16 (notably some Asian languages) require access to an alternate address space to identify the glyph for the character value. This requires the use of surrogate characters. These are pointers to code points outside the BMP. The range U+D800 - U+DBFF and U+DC00 to U+DFFF are reserved for this purpose; the surrogate character is a 32 bit value containing a pointer in both ranges to define the code plane and the specific code point.

Encodings are Important

When a computer displays a character, the process is:

1. Read the value.
2. Select the correct encoding scheme.
3. Look up the display character (glyph).

4. Display the glyph.

If the encoding system is interpreted incorrectly, the wrong glyph will be retrieved and presented. The result will be that the display is garbled.

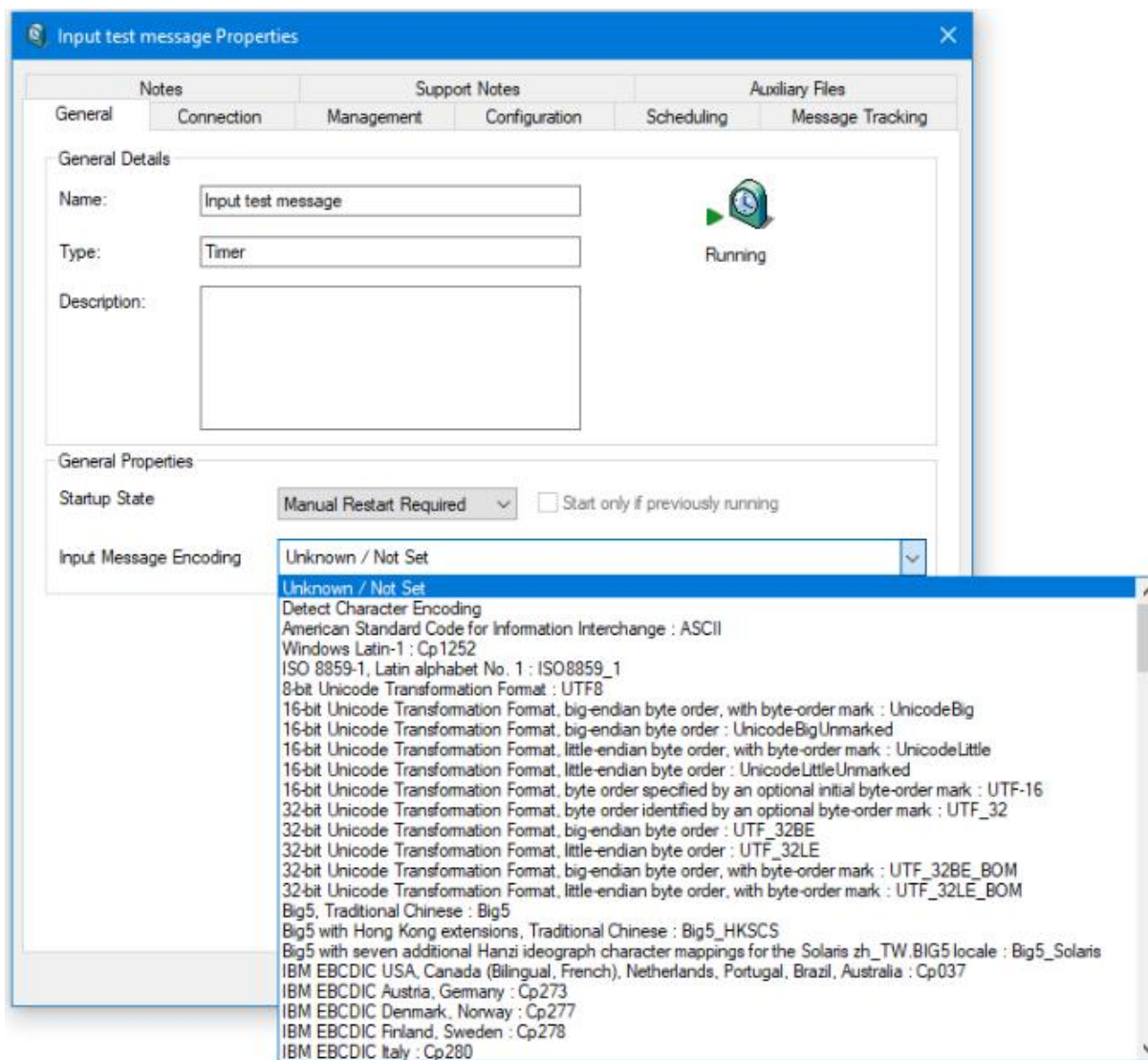
Rhapsody impact

The impact on the message in Rhapsody is similar. If the incorrect encoding scheme is selected, the value retrieved will be incorrect, and it may not be possible for Rhapsody to correctly interpret field values or the message content. For example, at a gross scale, the message may be interpreted as a stream of 8 bit values (UTF8), when in fact it should be interpreted as 16 bit values (UTF16).

Rhapsody and Encodings

Where messages processed through Rhapsody will have their contents interpreted or mapped, it is important that the correct encoding scheme is specified. We can specify encoding for a message in Rhapsody on the Input Communication point where it is received.

It is possible to specify the encoding of incoming messages on any communication point that has been configured in a mode that could receive a message (this is any mode except Output mode). This configuration is performed from the **General** tab of the communication point's **Properties** dialog.



The default setting for the **Input Message Encoding** setting is **Unknown / Not Set**. When this value is set, Rhapsody will use the language and locale settings of the Operating System to guess the appropriate message encoding. Frequently this will be [CP1252\(opens in a new tab\)](#) on Windows, or UTF8 on Linux. If the **Detect Character Encoding** option is selected, Rhapsody will inspect the first 100 characters in the incoming messages looking for clues to the correct encoding. This includes looking for a [Byte Order Mark\(opens in a new tab\)](#) and an XML prolog encoding details. It is also possible to specify any character encodings supported by Java.

Note that setting the encoding explicitly in the communication point configuration does not change the encoding for the message (that is, it doesn't translate it to the specified encoding); rather, it defines the encoding to use when displaying or extracting the content of the message.

Detecting HL7 message encoding

Rhapsody includes the HL7 Detect Character Encoding filter to detect the character encoding on HL7 messages. This filter looks at the contents of the MSH.18 and MSH.20 fields in an HL7 message, along with checks on the Byte Order Mark.

Using Unicode values in a filter

When a Unicode value needs to be specified in a filter, Rhapsody uses the Java notation `\uxxxx`; that is, `\u` followed by two hexadecimal characters. These values will need to be sourced from the reference table appropriate to the encoding in use.

For example:

- `\u0041` is A (Latin Capital A)
- `\u0061` is a (Latin Lower Case a)

Message Properties

While Rhapsody receives the body of the message from an Input communication point, it is frequently useful or necessary to define additional data about the message. This may be achieved by defining message properties, which are saved along with the message as meta-data. Any change in a property value will be written to the datastore in the same way as changes to the message body are written.

Properties may be defined:

- Automatically by the engine; for example, the following are defined for all messages:
 - `InputTime`
The timestamp for receipt of the message, expressed as the number of milliseconds since the Unix / Java Epoch (1 January, 1970)
 - `rhapsody:InputCommunicationPoint`
The name of the communication point on which this message was received
 - `rhapsody:InputCommunicationPointId`
The internal ID of the communication point which received this message
- Explicitly when the message enters the route and is parsed against a message definition (defined on the route properties dialog).
- Explicitly by filters on the route pipeline.

Generally, properties which are automatically defined by the engine are tagged with the **rhapsody** namespace tag (for example, rhapsody:InputCommunicationPoint).

△ Its very easy to create lots of message properties in your configuration. Only create properties when necessary. Having too many message properties can be overwhelming and confusing. Especially when viewing messages in the Management Console. Unneeded properties can be deleted by giving them a value of null.

Never store the message body or any data that is large in a message property. The datastore assumes that message properties are relatively small. Storing large data in to them will adversely affect the datastore. Better methods for retrieving a past version of the message body are discussed later in this course.

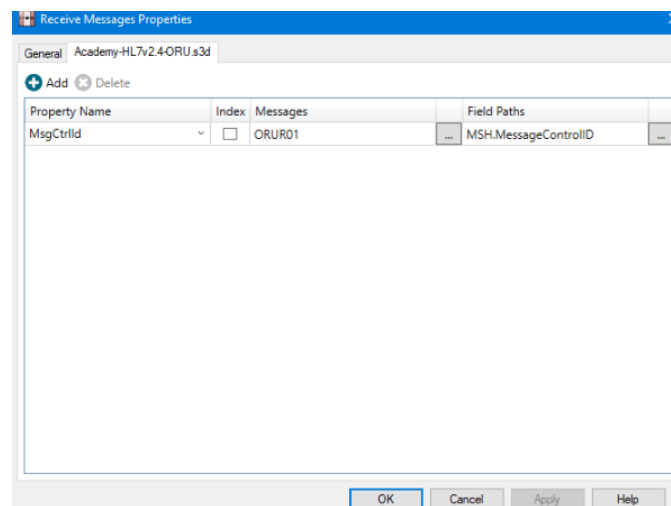
Once a message property has been defined, it remains valid and is accessible throughout the life of the message. Each property may be retrieved or updated by filters, as required. The values are retained across the route boundaries so that if the message passes through a number of routes, all the properties defined on any route it has passed through will be available.

Indexed properties

Although the message properties are displayed in the Rhapsody Management Console on the **Message View** page at each processing step, properties are not normally available for inclusion in a search condition.

Where it is desired to use a message property (for example, a patient identifier) in a search condition, the property must be defined as an **Indexed Property** in the route's **Property** dialog. This requires that:

1. There is a message definition attached to the route.
2. The property is defined in the Content Extraction tab for the message definition.
3. The property is marked for Indexing.



It is also possible to create an Indexed property within a JavaScript filter. This will be discussed more later in this module.

Indexed properties are saved to the Datastore for use as search criteria when the message exits a route. This means that:

- The property value may be changed as frequently as required as the message traverses the route. The permanently stored value made available for searching is the value when the message leaves the route.
- The intermediate values will not be located if included in a search; only the value as the message exits the route will be able to be located.
- The property value need not be set when first defined; it should however be set on the route.
- If the message passes through multiple routes, the property value will be saved to the Datastore each time the message leaves a route.

In order to prevent unnecessary processing and disk utilization by the message property indexing logic, ensure indexing is restricted solely to fields that are required for fast lookups on message searches.

The Message Body

The message body comprises the content of the message, initially the content as it is received by the Input communication point. As it passes through the route pipeline, it may subsequently be modified by the filters.

Every time the message is processed by a filter, an update is made to the datastore to track the message's progress. If the message body is altered, this is also written to the datastore. The set of message properties are written to the datastore anytime there are any changes. Some care is required to reduce the number of times a message and its properties are written to the datastore as this may have an impact on storage volumes and potentially on performance. This is normally done by using the smallest number of filters required to complete the configuration.

Message Structure

In the first instance, Rhapsody treats a message as an unstructured chunk of data. The engine is therefore equally able to handle binary messages as text and structured text messages.

In the absence of a message definition, Rhapsody will attempt to process a message that requires parsing as XML. If the engine is requested to address a message field or extract a value, field information will be interpreted as an XPath specification. An error will occur if the message cannot be parsed as XML.

Content extraction

Where processing logic requires that values be extracted from a message, two approaches are available:

1. The message can be treated and manipulated as text.
2. A message definition allows the structure and content to be addressed. This requires construction of a Rhapsody Messaging Toolkit definition file (.s3d) or an XML Schema (.xsd) as appropriate.

Implicit Structures

Rhapsody makes no assumptions about the content of a message and is therefore able to accept and handle binary messages as well as messages containing text or structured text. However, the ability to derive values or otherwise manipulate the message depends on the ability to extract portions of the message for processing; in other words, to define the message structure to the engine.

Rhapsody is able to handle implicitly structured messages in XML and JSON formats as well as explicitly structured messages where a Rhapsody Messaging Toolkit definition file is provided to define the structure.

This module reviews the character and handling of implicit and explicit message structures.

XML Messages

XML messages are structured **eXtensible Markup Language** messages which are designed to have both a machine and a human-readable structure. Rhapsody provides implicit support for parsing such messages and building the Document Object Model (DOM) for the message required to address fields and manipulate the structure.

The example below shows an XML message with an indented and highlighted structure to help illustrate its format.

An XML structure contains nodes, each of which may contain values, attributes, or child nodes.

Each node is denoted by tags which mark the start and end of the node data; the tags are defined by enclosing the nodename in < and > markers, and the closing tag includes a / before the nodename. In the example above <message> denotes the opening of the **message** element, and </message> marks the close. An empty node may be marked with a single tag; for example, <source_file null="yes"/> denotes that the source_file node has no content (strictly, that it has an attribute null with the value yes).

Nodes in an XML structure may be nested to any depth, and it is possible to determine the value of a node by walking down the node tree. The desired (target) node is identified by the XPath description of the node. For example, the `id_org` node in the example above can be identified as `/message/recent/id_org`.

A node may be repeated many times in the structure; a required repeat may be identified using array notation (`[n]` where `n` is the one (1) based iterator). Each node (including its children) is described as an element in the structure.

Well-formed XML

A well-formed XML structure contains:

- An XML Declaration (`<?xml version="1.0" encoding="UTF-8"?>` in the message above).
- A root (top level) node with opening and closing tags which contain all other nodes in the structure
- Child nodes, each fully enclosed in the immediate parent node and nested to any depth, and likewise denoted with opening and closing tags
- Nodes that each may contain:
 - A value, defined between the opening and closing tags.
 - One or more attributes, defined by name / value pairs within the opening node tag with the value quoted; for example `<note id="10">` defines the element **note** with an attribute **id** which has the value 10 for this element. Attributes are often utilized to define metadata about the element rather than values.

An XML value may not contain the characters `<`, `>`, and `&` as these are structural elements. If these are required in values, the sequences `<`, `>` and `&` should be used.

A useful reference source is [w3schools.com/xml](https://www.w3schools.com/xml/)(opens in a new tab) and for XPath detail, [w3schools.com/xpath_intro](https://www.w3schools.com/xpath_intro/)(opens in a new tab).

XPath Field Addressing

The structure of an XML message can be determined and a Document Object Model (DOM) built for it by traversing the nodes in the message and building the structure. Rhapsody provides support for this utilizing the Apache [Xerces](https://xerces.apache.org/)(opens in a new tab) component.

The message DOM will be constructed on demand to permit XPath addressing to be used to extract required node values. This is particularly useful in the JavaScript environments (conditional connectors and filters) as well as the database filters.

Fundamentals

XPath element addressing notation is most commonly utilized in XSLT transform processing instructions; a sub-set of the functions is used for field addressing.

The key components of the XML structure which need to be identified are:

- Nodes
The structural elements of XML, containing attributes, values, and child nodes.
- Attributes
Meta-data about the nodes. They should not carry value data as attributes.
- Text values
The content of the element (that is, the element value).

The path to the required element is specified by listing the names of the nodes to be traversed in order, separated by the forward slash (/) character.

The path may be specified as:

- Absolute
Specified by including a leading /; an absolute path must include the full path from the root node to the element required.
- Relative
Specified by a leading // (double slash); the path specification may start at any level in the tree - selection will start from the first location matched by the path expression.
- Context path
If the leading / or // is omitted, the path is interpreted as the absolute path within the current context; for field and value selection, this will be from the root of the message.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<message>
```

```
<recent>
```

```
<pi_index>0</pi_index>
```

```
<source_file null="yes"/>
```

```
<modified_date>2012-09-07 09:53:46.976883+12</modified_date>
```

```
<modified_by>Service</modified_by>

<last_name>LOFTIS</last_name>

<id_org>CANON</id_org>

<patient_id>8</patient_id>

<given_name>KAYA </given_name>

<lastaccess>2012-09-07 09:53:52.425277+12</lastaccess>

</recent>

<recent>

  <pi_index>0</pi_index>

  <source_file null="yes"/>

  <modified_date>2012-09-07 09:53:46.976883+12</modified_date>

  <modified_by>Service</modified_by>

  <last_name>BOTHEL</last_name>

  <id_org>CANON</id_org>

  <patient_id>8</patient_id>

  <given_name>KINFE </given_name>

  <lastaccess>2012-09-07 09:53:52.425277+12</lastaccess>

</recent>

</message>
```

Examples

Expression	Result	Comment
<code>/message/recent[2]/last_name</code>	BOTHEL	Absolute path to the element last_name in the second repeat of recent.
<code>//recent/last_name</code>	LOFTIS,BOTHEL	All values of element recent/last_name, irrespective of where in the tree the element recent is located; comma separated list.
<code>//recent[1]/source_file/@null</code>	yes	The value of the attribute null for the element source_file in the first iteration of the element recent.
<code>/message/recent[3]/last_name</code>	<i>empty string returned</i>	Iteration 3 of the recent element does not exist.

Element values

Element values are normally parsable text (that is, alphanumeric data not containing the special characters <, > and &). This is referred to as **PCDATA** in the formal definition of an XML structure. Note that the element may be defined as having non-parsable character data which may contain the special characters as an XML processor will not attempt to process this content; this is referred to as **CDATA** in the formal definition of an XML structure.

Some caution is required when defining an XPath expression as a non-existent path will return an empty string ("") rather than cause an error to occur.

Explicit Structures

Explicit message structures are defined by explicitly associating a definition document with the message which will enable it to be parsed; this requires either a Rhapsody Messaging Toolkit message definition (.s3d) or an XML Schema definition (.xsd).

Message definition files are built externally to the Rhapsody environment and then imported to the engine. Once they are available to the engine, they will be listed in the Definitions tab in the workspace and may be associated with routes and some filters. A message definition may be re-used multiple times throughout the configuration.

When a message definition is associated with a route, the message will be parsed according to that definition as it enters the route and the message object will be

constructed. Note that the message will not be parsed if it is likely that the message object is not required during the message's path through the route; notably, such as if there are no filters on the route, for example.

The creation of message definitions using the EDI Message Designer and the Generate XSD utility were extensively covered in the ***Rhapsody Associate*** course. It is recommended that you review that content before proceeding.

EDI Explorer

The EDI Explorer is a tool that allows messages to be inspected against a message definition created in EDI Message Designer. It is frequently used to test definitions, confirming that they are able to process messages without error. The tool is also used to inspect messages against a message definition and either modifying them for testing purposes or reporting identified errors to a third party.

Open EDI Explorer from within the Rhapsody IDE by selecting **EDI Explorer** from the **Tools** menu. It can also be opened from EDI Message Designer by selecting **Launch EDI Explorer** from the **Rhapsody Messaging Toolkit** menu. When opened this way, the current message definition is automatically loaded into EDI Explorer.

Load a definition

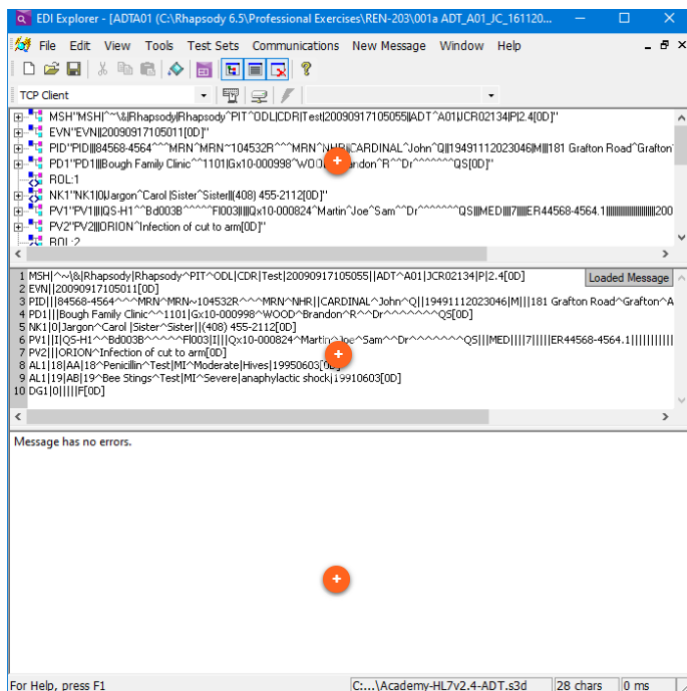
Load a definition by selecting **Open Definition** from the **File** menu. The **Recent Definitions** list, also underneath the **File** menu, contains the ten most recently used definitions. If frequently working with one or more definitions, the recent list will save time.

Load a message

Load a message by selecting **Open** from the **File** menu. The **Recent Messages** list contains the ten most recently viewed messages. To open a sample message, you will need to select **All File Types** in the browse dialog to view the available files.

Inspect and edit message

When a file is loaded, three panels will be displayed for that message.



A field can be edited by expanding the parsed content to select the appropriate field, then double-clicking on it to edit the field value. A red border will appear around the file content for the selected field. All three panels will be updated as the message is edited.

Where there is a repeating field or segment in the message, a new repeat is added by right-clicking on the appropriate element and selecting **Add New Repeat**.

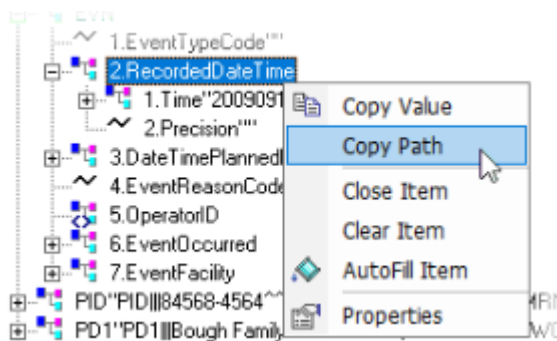
Double-clicking on an error in the error panel will select the appropriate field ready for editing.



Get a field path

Sometimes there is a requirement to record the path of a particular field. For example, when using a JavaScript filter to get or set fields in the message or when sending an email containing detailed message information to a client.

To determine the field path, right-click on a node in the message structure and select **Copy Path** from the drop-down list. This will copy the selected path to the clipboard. For example, in the screenshot below, EVN/RecordedDateTime would be copied to the clipboard.



Create a new message

A new message can be created by selecting the appropriate message type to create from the **New Message** menu. The EDI Explorer provides the **Message Autofill** tool to populate a new message with more or less sensible data in each field, and which therefore doesn't encroach on Patient Privacy requirements.

Using the Autofill tool (**Tools / AutoFill Messages**) also permits the user to build sets of messages with a range of message types (as defined and selected from the **Message Definition**) with multiple messages per type. This is frequently valuable in the initial testing phase when more accurate test messages are not available. Selected parts of a message can be autofilled by right-clicking a message node and selected **AutoFill Item** from the drop-down list.

Send a message via TCP

EDI Explorer includes an inbuilt TCP client. This can be used to send and receive messages from other systems, including a TCP Server communication point. This is used when a test message in EDI Explorer needs to be sent to a TCP server as part of a system test.

To configure the TCP client, select **Communications Setup** from the **Communications** menu.

The **Communications** section will need to be set up appropriately for the TCP server that is being connected to, while the **Wrappers** and **Character Encoding** sections will normally be correct with the default settings. Save changes when done.

To connect to the server, select **Connect** from the **Communications** menu. To send the message, select **Start Sending** from the same menu. Any response message received from the server will be loaded into EDI Explorer as a new message.

Save a message

A message that has been created or modified can be saved to disk by selecting **Export Flatwire Message To File** from the **File** menu. The **Save** option should not be used as it saves the file in a format used exclusively by EDI Explorer.

The Mapper

The Mapper translates messages between any of the following message definitions:

- Message definitions created using the EDI Message Designer (such as an HL7 v2.4 definition).
- XML messages with an XSD definition.
- JSON messages with a [JSON Schema\(opens in a new tab\)](#) definition (such as the [FHIR JSON\(opens in a new tab\)](#) definition).

The *Rhapsody Associate* course introduced basic use of the Message Mapper. This module will cover the usage of the mapper in more complete detail. You may choose to review your notes for *Associate* before proceeding.

Creating New Maps

Main maps and sub maps are created and edited in a number of ways.

Drag and drop

Items can be mapped together by dragging the source element onto the intended destination element.

New Map

Map Name:

Variable Datatype	Variable Name
Input::ADTA04	in
Output::notification	out

Preview:
map MapADTA04T onotification(Input::ADTA04 in, Output::notification out)

Automap
☐ Automatically map
 Map by:

☒ Open the new submap for editing

Select **Add Parameter** to add additional parameters to the map. For example, a set id for a repeating group of elements. The parameter type is selected from the drop-down in the **Variable Datatype** column.

Map Parameters

Variable Datatype	Variable Name
Input::ADTA04	in
Output::notification	out
Param	

Choose message structures by selecting the **Browse** option. Any variable with a datatype prefixed with Input will be an input to the sub map, while those prefixed with Output will be an output parameter. A suitable name for the parameter should be entered into the **Variable Name** column.

Add a new sub map manually

Add a new sub map to a mapping by selecting **Mapping > New Sub Map**, or by right-clicking on the **Workspace** and selecting the option that way. This opens the **New Map** window (as above).

Manually creating a sub map is useful when creating a map that performs an auxiliary task (such as escaping text), or when complete control is required over the definition of the sub map.

A suitable name must be entered into the **Map Name** field. By default, this field contains the text "MapTo" which is typically a part of all sub maps (although not needed if the sub map is not mapping one or more complex structures). Input and output parameters will also need to be added to the definition.

Update an existing definition

An existing sub map definition may be edited either by right-clicking on the name of the map in the **Workspace** and selecting **properties**, or by right-clicking in the editor window and selecting the **Edit Definition** option. This will open the **Edit Map Definition** window that allows for the editing of parameters associated with the definition (as above).

Add a new main map

It is possible to add more than one main map into a map definition. Where multiple similar message types (for example, different HL7 ADT messages) need to be mapped, this allows the various sub maps to be reused, reducing effort and improving the consistency of the output messages. The only requirement is that all input messages and all output messages share single message definitions.

To add a new main map, either select **Mapping > New Main Map**, or right-click on the **Workspace** and select the option that way. This will present you with a list of input and output messages. This is the same list that is presented when the mapping is first created.

Automap

When creating a new sub map, Automap options are included on the **New Map** screen. The Automap option is able to automatically map input field values to their corresponding field in the output message.

Note that **EXTREME CARE** should be taken when using the Automap feature as the feature is very limited in its ability to match input and output fields that should be mapped together. In general, the Automap feature should only be used when the input and output message definitions are very similar.

When a sub map is created with the Automap feature turned on, a careful review of the resulting code should be performed to make sure it meets the mapping requirements.

Any unmapped inputs or outputs will appear at the bottom of the sub map inside a TODO comment. For example:

```
/* TODO Unmapped outputs could not be automatically mapped:
```

```
    idType
```

```
    idNumber
```

```
    idNamespaceID
```

```
*/
```

Items that are manually mapped, or deemed to not require mapping, should be removed from the TODO list to avoid confusing anyone who might look at the sub map in the future.

Where a map definition is making modifications to an HL7 message and a significant number of the sub maps could use the Automap function, consider using the **Message Modifier** filter (discussed later in the module) instead.

Mapping Repeating Fields

Where the input or output message definition contains a repeating field, we can use a number of approaches to perform a mapping. These approaches let us map a specific item, map all items, or add an item to the end of a repeating field.

Map a specific item

By selecting a specific index (for example: 0), the mapper will map just that repeat from the input map and ignore all other repeats of the input field.

For example, the following code maps the first instance of the PatientIdentifierList input field to the corresponding output field.

```
MapCXToCX(in.PatientIdentifierList[0], out.PatientIdentifierList[0]);
```

Map all items

Entering an index of * (asterisk) into the input and output fields causes the mapper to map all repeats of the input field to the output.

The following code will map all repeats of the patient's PatientIdentifierList to the output field.

```
MapCXToCX(in.PatientIdentifierList[*], out.PatientIdentifierList[*]);
```

✎ The following for loop functions the same as the star operator above.

```
for (int i = 0; i < sizeof(in.PatientIdentifierList); i = i + 1)
{
    MapXCNToXCN(in.PatientIdentifierList[i], out.PatientIdentifierList[i]);
}
```

Append an item to an output array

It is possible to use a plus operator on an **output** field to append the contents of an input to it.

For example, the following code would add the contents of the PatientIdentifier field to the output PatientIdentifierList field.

```
MapCXToCX(in.PatientIdentifier, out.PatientIdentifierList[+]);
```

⚠ Care needs to be taken to follow best practices when using the plus operator. If we attempted to map the following code using the plus operator:

```
out.patientName[+].givenName = in.name.first_name;
out.patientName[+].familyName = in.name.surname;
```

The resulting message would contain the given name if the first repeat of the patient name, and the family name in the second repeat (this is generally not what we want). To fix this, we need to remember the mapper best practice of mapping one level of the message at a time. The following mapper code does this:

```
MapNameToPatientName(in.name, out.patientName[+]);
```

Checking for and Managing Field Existence

Any field in a message could contain data, otherwise the field might not exist in a particular message, or it might exist but contain no content (either empty or null). This section deals with detecting and processing non-existent and empty fields.

The following table details the difference between the various field states:

Field state	Description
Does not exist	A field (or path) that does not exist in the message.
Empty	A field that is empty exists in the message, but contains no value. In many messaging systems, non-existent and empty fields should be treated as being equivalent.
Null	A null value is a special value to indicate that no information exists for a field. This is different from an empty (or non-existent) field. In HL7, a null field is indicated with a pair of double quotes in a field. A null value will often be used in a mandatory field to indicate that no information exists for the field.
Contains Content	A field that contains some non-null information.

Checking for Existence

The `exists()` function should be used to check any path in a message to see if it exists prior to evaluating its content. The function returns true if the specified path exists, otherwise, it returns false.

```
// Map Procedures
```

```
if (exists(in.Section[i].Procedure)) {
    MapProcedureToPR1(in.Section[i].Procedure, out.PR1[+]);
}
```

When checking fields to see if they contain a value, the `exists()` function should be used first to ensure the field exists.

```
if (exists(in.AssigningAuthority.NamespaceID) && in.AssigningAuthority.NamespaceID
!= "NatId") {
    out.IDNumber = in.AssigningAuthority.NamespaceID;
}
```

This prevents errors from occurring when attempting to check the contents of the field does not exist.

Managing Empty Fields

The `isempty()` function will return `true` if the field is empty or does not exist in the message. The function will return *false* if the field contains content or contains a null value.

```
if (!isempty(in.MiddleInitialOrName)) {  
    out.SecondName.#PCDATA = in.MiddleInitialOrName;  
}
```

Managing Null Fields

The `isnull()` field returns `true` if a field contains a null value, otherwise, if the field contains some content or is empty or non-existent, it will return *false*.

```
string addressMsg;  
  
if (isnull(in.address.StreetAddress)) {  
    addressMsg = "No address supplied";  
} else {  
    addressMsg = in.address.StreetAddress;  
}
```

Use the `setNull()` function or the null special variable if a field in a message need to be set with a null value.

```
if (!exists(in.address)) {  
    out.address.StreetAddress = null;  
}
```

Determining the Size of a Field or Object

The `sizeof()` function returns the length of a field, or if given the path of a repeating element in the message structure, of the number of repeats of that element.

```
string reportComment = "There are " + IntToStr(sizeof(in.OBX)) + " observations in this message";
```

The sizeof() function can be used to select fields that only contain non-null content. Any null, empty or non-existent field will return a size of zero.

```
if (sizeof(in.Recipient.#PCDATA) == 0) {  
    out.ReceivingFacility.NamespaceID = "Unknown";  
} else {  
    out.ReceivingFacility.NamespaceID = in.Recipient;  
}
```

Mapping Groups of Elements

Sometimes there is a desire to map a group of elements without creating the submaps needed to map field by field, or to manipulate a group of fields as if they were a string (for example, the HL7 date fields). In this case, we can use the getEncodedValue() and setEncodedValue() mapper functions.

The getEncodedValue() operator takes a single parameter containing path to a message item and returns a string containing that item's encoded value. For an XML message item getEncodedValue() retrieves the XML string of that item.

The setEncodedValue() statement is used to set the contents of an output message item. The function takes two parameters; the first is the path of the item to set. The second parameter is the encoded value itself. It allows the user to set the contents of a message item in one step rather than setting the values individually, and in conjunction with getEncodedValue() allows bulk copying of identical structures byte by byte, including message delimiters. This function cannot be used on XML documents.

Examples

Map the contents of the MSA segment in the input message to the corresponding segment in the output message without creating sub-maps:

```
// Map MSA Segment
```

```
setEncodedValue(out.MSA, getEncodedValue(in.MSA));
```

Retrieve the message date time from the MSH segment:

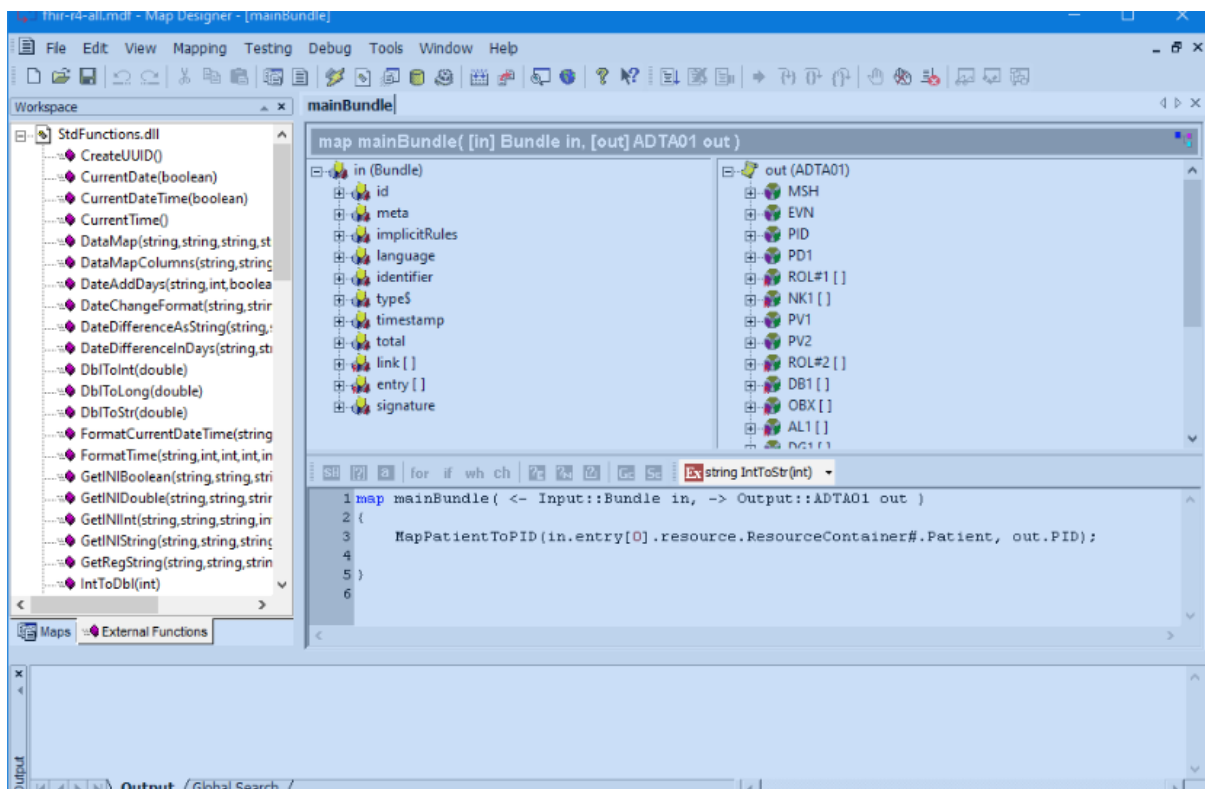
```
// Get the message date
```

```
string msgDate = getEncodedValue(MSH.DateTimeOfMessage.Time)
```


External Functions

There are a number of useful functions provided in the mapper for performing tasks that may be complex or may be too difficult to do with a sub map. These are referred to as **External Functions**.

External Functions can be accessed with the **External Functions** tab in the Mapper **Workspace**. They can also be accessed from the ribbon above the Code Window.



When the desired function has been found, it can be dragged from the External Functions tab into your code or inserted by selecting the function name in the code window ribbon.

Right-clicking on a function name in the Workspace allows you to view the function's properties. The properties give a brief description of the function and its parameters.

Commonly Used Functions

External Functions can be split into four main groups:

- String manipulation
- Data type conversions
- Date/Time manipulation
- Miscellaneous

String Manipulation

The most common types of functions perform string manipulation. This allows great flexibility when having to transform or perform complex logic when faced with tasks that can't be performed using programming code. Some of the string functions that can be performed are:

External Function	Description
<code>StrToUpper()</code> / <code>StrToLower()</code>	Converts the string to upper or lower case.
<code>StrRemoveChars()</code>	Removes unnecessary characters from the string.
<code>StrRegFind()</code>	Allows using a Regular Expression to locate a pattern in a string.
<code>StrTrim()</code>	Removes whitespace from the beginning and end of a string.
<code>StrMid()</code>	Retrieves a sub-string from within a string.

Data Type Conversion

Messaging protocols such as HL7 or X12 frequently have different datatypes to represent field values. Data types might include, string, numeric (integer, double, long, float), and Boolean.

Some EMR (Electronic Medical Record) systems express all data types as string. It may be necessary to convert a numeric data type to a string before sending to the EMR or converting the data from the EMR to numeric before sending to an external system.

Some common data type conversions are:

External Function	Description
<code>StrToInt()</code>	Converts a string value to numeric integer.
<code>StrToDbf()</code>	Converts a string value to numeric double.
<code>IntToStr()</code>	Converts a numeric integer to string.

Date/Time Manipulation

There are several functions that allow the creation or formatting of the date, time, or both. For example, if you need to convert an HL7 format date from to an ISO 8601 formatted date in an XML message.

External Function	Description
<code>FormatCurrentDateTime()</code>	Return a formatted string containing the current date. The format string uses the strftime formatting codes.
<code>FormatTime()</code>	Takes the various date parts as a series of integers and formats them into a date/time.
<code>DateChangeFormat()</code>	Takes a date in one format and converts it to another. The function includes special format keywords for HL7 and ISO-8601 date formats.

The date format functions help simplify your code by eliminating the messy string manipulation otherwise needed to format a date.

In addition to the functions above, there are a number of date arithmetic function that can be used to add days to a date or calculate the difference between two dates. Date/Time manipulation is discussed in more detail on the next page of this lesson.

Miscellaneous

There are some functions that are not specific to previously mentioned groups. Some of these functions are:

External Function	Description
<code>RhapsodyTableLookup()</code>	Used for accessing a Rhapsody Lookup Table for translation. This function is only available when the Mapper has been launched from the Rhapsody IDE.
<code>CreateUUID()</code>	Creates a Universally Unique Identifier.

External Functions

Date Handling Examples

Examples of the three most commonly used date time functions are detailed below.

FormatCurrentDateTime(format)

This function uses the supplied format to return a formatted string containing the current date. The format parameter indicates the format of the resulting date. This string uses strftime formatting codes.

This example generates a human-readable representation of the current date:

```
string currentDate = FormatCurrentDateTime("Today is %A, day %d of %B in the year %Y.");
```

This generated string might contain this: Today is Thursday, day 16 of July in the year 2020.

In the following example, we format the current date-time as a HL7 formatted string and insert it into the out.DateTimeOfMessage field of the message.

```
setEncodedValue(out.DateTimeOfMessage,  
FormatCurrentDateTime("%Y%m%d%H%M%S"));
```

The setEncodedValue() function allows us to update all the parts of the date in the message together.

FormatTime(format, year, month, day, hour, minute, second, milliseconds)

This function takes the various date parts as a series of integers and formats them into a date/time. The format string uses the [strftime\(opens in a new tab\)](#) formatting codes.

```
out.time = FormatTime("Today is %A, day %d of %B in the year %Y.", 2012, 10, 23, 4, 0, 34,  
675);
```

DateChangeFormat(date, currentFormat, newFormat)

This function takes a date in one format and converts it to another format. The format parameters for this function are different to the format parameters use for other date functions. The date change function includes two special format keywords for HL7 and ISO-8601 formatted date (as commonly used in XML and JSON messages).

A HL7 date would be converted to a ISO-8601 date with the following code:

```
out.#PCDATA = DateChangeFormat(getEncodedValue(in.Time), "HL7", "ISO");
```

The getEncodedValue() function is being used here to return all the parts of the HL7 date as a single string. When using this function to update an HL7 message, the setEncodedValue() function would be used to update all the parts of the date together:

```
setEncodedValue(out, DateChangeFormat(in.time, "ISO", "HL7"));
```

A human-readable version of a date in an HL7 message could be generated with the following code:

```
string details = DateChangeFormat(getEncodedValue(in.AdmitDateTime), "HL7", "d  
MMM YYYY") +  
    " at " +  
    DateChangeFormat(getEncodedValue(in.AdmitDateTime), "HL7", "h:mm");
```

The output from this example could be 24 May 2015 at 2:27PM. The DateChangeFormat function is called multiple times as we can't add non-formatting characters within the format string. We have split the statement across multiple lines to make it easier to read (the hanging indent help identify all the lines as belonging to the same statement).

Global Variables and Message Properties

The Mapper supports globally declared variables that can be accessed from any main or sub map. This is useful when a constant value is required in a number of places within a map, such as a version string. It is also useful when a variable is needed to keep track of some value across multiple functions, such as when tracking the set Id across repeating OBX segments.

To create a global variable, open the **Global Declarations Code** panel from the **Mapping** menu. All variables created here must be prefixed with the global key word.

```
global int OBXSetId = 1;
```

```
global string SendingSite = "CityHospital";
```

```
global string SendingApp = "Laboratory";
```

Global variables within a function are accessed in the same way as locally defined variables.

Accessing Message Properties

Message properties associated with a message can be accessed in the Mapper by creating a global variable in the map of the same name as the message property and with no value assigned to it.

For example, if there was a need to access the BaseFilename and Suffix message properties from the **Mapper**, the following code would be added to the **Global Declarations** panel.

```
global string BaseFilename;
```

```
global string Suffix;
```

Best Practice

The following best practice guidelines help the creation of easy to understand and easy to maintain maps.

Structure

Structure the map and submaps by the output map requirements. This is naturally the way that the mapper constructs statements (especially when the Automap function is used). Consequently, structuring by the output requirements will result in a more consistent map definition.

Errors in the map are typically reported as they are found in the output map. Structuring the map this way makes the errors easier to find and fix.

Comments

Comment liberally, particularly changes and updates. There is nothing worse than reviewing a map definition that someone else has created or a definition that was created many months ago to discover that there are no comments in the source. Anything in a map that is not immediately obvious should have a comment. Likewise, any change or update to an existing map should be commented. If the change relates to an issue tracked in a management system, the issue number should be included in the comment.

Submaps

Make use of submaps liberally - you don't know when you might need them again. Creating submaps allows sub-sections of the input message to be mapped to the equivalent section in the output message repeatedly and consistently.

Submaps improve the readability of the map by removing the repetitive detail associated with each submap.

Good:

```
MapXPNTToName(in.PatientName[0], out.name);
```

Bad:

```
out.name.title.#PCDATA = in.PatientName[0].Prefix;  
out.name.givenName[+].#PCDATA = in.PatientName[0].GivenName;  
out.name.givenName[+].#PCDATA = in.PatientName[0].MiddleInitialOrName;  
out.name.familyName.#PCDATA = in.PatientName[0].FamilyName.Surname;
```

Map One Level at a Time

Constrain mappings to one level. If you require a lower level, use a submap. This helps break the map into manageable chunks and supports the reuse of submaps.

Naming Conventions

Establish naming conventions for your variables and submaps before you start. This will result in a map definition that is easier to understand, both now and in the future.

Testing

Always compile and test your maps before finishing. Many basic errors will be caught by quickly testing a map prior to using it. A map can be compiled by selecting **Compile Mapping** from the **Mapping** menu, or by pressing **F7**. This confirms that the input and output definitions are correct and there are no errors in the structure of the map code. A map can be tested by selecting **Run Test** from the **Testing** menu or by pressing **F5**. Testing of maps is discussed later in this module.

Where a definition has been created or modified, the **EDI Explorer** should be used to test the definition and confirm that test messages are being parsed correctly.

Configuring the Mapper filter for Error Trapping

A map will sometimes generate an error that is not detected during testing. By default, the Rhapsody Mapper filter does not trap or log errors.

The **Fail On Errors** property of the Mapper filter should therefore be changed from the default value of false to a value of **true**. This ensures that any error that might occur when processing a message (and may result in the loss of clinical information) ends up on the Rhapsody error queue for follow-up.

The **Log Errors** option should also be set to **true**, although it will result in numerous warnings about message structure if the message definition doesn't fit the message content; this should prompt revision of the message definition to reduce the logging load.

Mapping Disparate Structures

The mapper is able to map messages between any two message definitions, including those created in **EDI Message Designer** and those which have an XML **XSD** definition.

For example, the mapper could be used to transform an HL7 2.6 ORM laboratory order message into an HL7 2.3.1 ORU laboratory result message, or to transform an HL7 REF referral message into an XML referral message. In both cases, a message definition will be required for both input and output message structures. In addition to this, a detailed understanding of the mapping to be undertaken is needed.

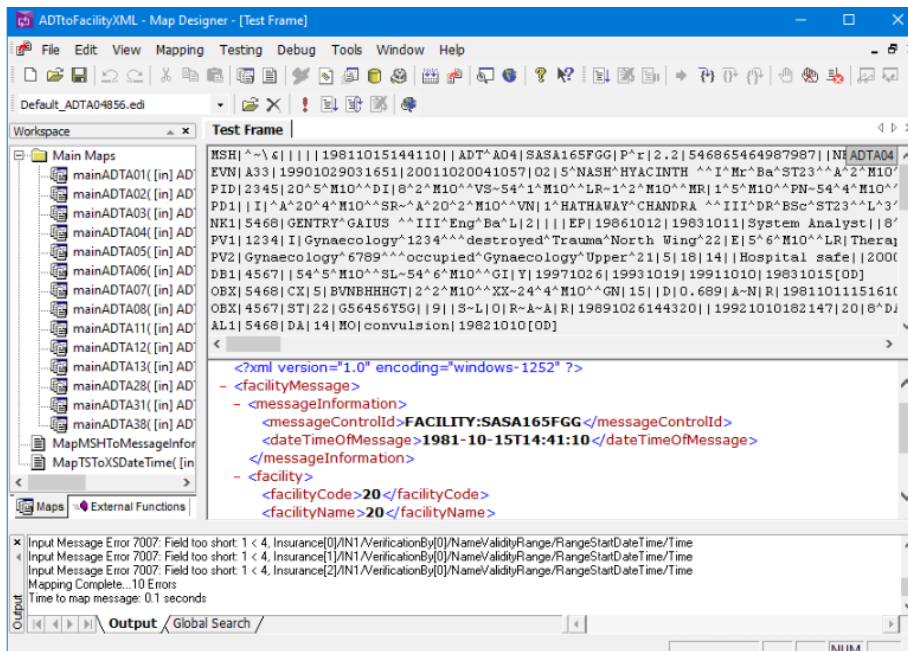
Considerations

When mapping a message between two different message structures, the following items need to be taken in to consideration.

- Many structured message definitions have fixed field lengths. The Mapper will let you put more content than permitted into a field; however, many receiving systems will reject a message with too much data. A decision as to what to do with any data that is too long for a given field in the destination message will depend on the importance of the data and the behavior of the target system.
- It is more likely that fields will need to be combined or (sometimes) split when mapping between very different message structures. These should be handled by submaps as it makes it easier to reuse the configuration as required and makes the main map and other sub maps significantly easier to understand.
- Be aware of any encoding or character set differences between the systems. While most encoding differences will automatically be managed by Rhapsody, some character set differences may require manual changes to text in the Mapper or a filter that performs post-processing of the out message. For example, a destination system may require that any units containing a caret symbol (^) are converted to an asterisk (*).

Debugging

The Map Designer contains a number of debugging functions to allow a mapping to be thoroughly tested prior to committing the changes in Rhapsody. To access the functionality, select **Run Test** from the **Testing** menu or press the **F5** key. You will be asked to provide a test input file the first time the debugger is accessed. Once a suitable input file is selected, the mapper will perform the test and show the result in the bottom half of the Test Frame.



Any errors associated with either the input, output message or mapping will appear in the output panel at the bottom of the screen. Errors in the output panel caused by problems in the mapping can be **double-clicked** to show the code that caused the problem.

Test Messages

Additional input test messages can be added into the Map Designer by selecting **Add Test File** from the **Testing** menu. The currently active test message is the message visible in the drop-down list just above the workspace. When selecting input messages for testing, they should be chosen to cover as complete a range of possible scenarios in as few messages as possible.

Test output messages are automatically saved to disk in an output folder under the mapper folder.

Break Points

Break points can be set on a particular line of code in the mapper to force the mapper to stop execution at a particular place in the code when running a test map. A break point is set by **right-clicking** on a line of code and selecting **Insert/Remove Breakpoint**. The break point can be removed following the same process.

```

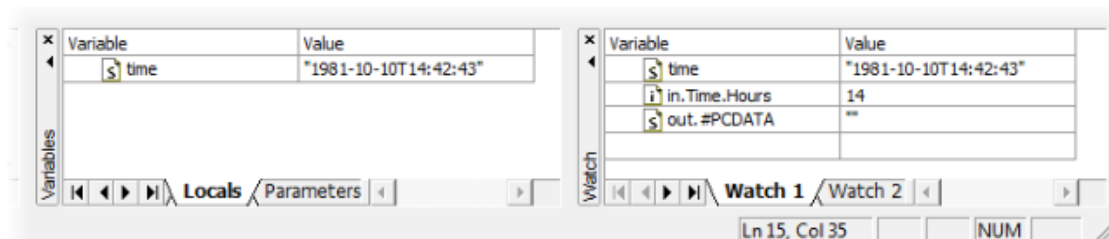
// Calculate the total length of the datetime string
int datetimeLength = sizeof(ts.Time);
// Populate the year
string time = StrMid(ts.Time, 0, 4);
// Populate the month if it is provided
if (datetimeLength >= 6) {
    time = time + "-" + StrMid(ts.Time, 4, 2);
}
// Populate the day if it is provided
if (datetimeLength >= 8) {
    time = time + "-" + StrMid(ts.Time, 6, 2);
}
// Populate the second if it is provided
if (datetimeLength >= 10) {
    time = time + "." + StrMid(ts.Time, 8, 2);
}
// Populate the second if it is provided
if (datetimeLength >= 14) {

```

When running a test with break points set, the mapping will pause at each break point encountered. The current position of the code execution is marked with a yellow arrow in the left margin. When the mapping is paused, pressing **F5** will resume mapping, or any of the options under the **Debug** menu can be used to step through the code.

Watch Variables

When a test mapping is paused in the debugger, it is possible to inspect the input and output messages along with all local and global variables.



Variables and Watches

The value of all local variables is shown at the bottom of the screen in the **Locals** tab of the **Variables** panel. The contents of input and output messages can be viewed from the **Parameters** tab of the **Variables** panel. The contents of the variables panel will change as the debugger steps through the code and the set of currently defined local variables changes.

It is also possible to quickly check the value of a variable when the mapper is paused by hovering the mouse over the variable of interest in the code window.

```

18     addressLine = addressLine + in.StreetName;
19 }
20
21 if(!isEmpty(in.StreetOrMailingAddress)) {
22     if(sizeof(addressLine) > 0) in.StreetOrMailingAddress = "99 Khyber Pass Road"
23     addressLine = addressLine + " ";
24 }
25
26 addressLine = addressLine + in.StreetOrMailingAddress;

```

Mouse hovering over the `in.StreetOrMailingAddress` variable to inspect its value

A set of watch variables can also be created that can cover any items from the input or output messages or from the global or local variables. From the **Watch** panel at the bottom of the screen, enter the name of the variable to be watched in the empty **Variable** column at the bottom of the list.

JavaScript –

Introduction

Rhapsody has many components that perform specific functions or manipulate messages in some way. At times, you may need to perform complex logic or transform a message that isn't possible with the built-in filters. This is where JavaScript is helpful.

An example is the creation of a message property that could contain a value from one of several fields in the message based on a number of conditions, such as different values from different fields. While you could use conditional connectors and property population filters, the route could soon become cluttered and difficult to maintain if there were many values.

Why JavaScript?

JavaScript was developed as a scripting language that controls the behavior of web pages in a web browser. It is a loosely typed programming language that is easy to learn.

- Loosely typed
JavaScript doesn't require variable types to be declared. In many programming languages, a variable data type must be specified before the variable can be used. "String" or "Integer", for example. In JavaScript, a variable is declared using "var".
- Easy to learn
Even if you have no programming experience, with a little practice you can learn JavaScript quickly and add more flexibility to your Rhapsody solutions. The web-

based scripting logic isn't necessary when using JavaScript in Rhapsody. JavaScript in Rhapsody is typically relatively simple.

JavaScript in Rhapsody

Rhapsody provides the ability to use JavaScript to assist with message processing and management. Some components in Rhapsody that use JavaScript are:

- JavaScript (v.2) Filter
- JavaScript Connector
- Directory Communication Point
- (S)FTP Communication Point
- JavaScript REST Client (v.2) Communication Point
- JavaScript TCP Client Communication Point
- JavaScript TCP Server Communication Point

Rhapsody introduced a new [JavaScript scripting engine\(opens in a new tab\)](#) in version 7. The new JavaScript engine made several changes including removal of support for E4X (a JavaScript extension used to process XML messages). To help manage site upgrades, v.2 versions of some communication points have been created. When creating new configuration, you should use the newer (v.2) components.

When using JavaScript within Rhapsody:

- JavaScript components are restricted by the interpreter to a maximum of 32KB of JavaScript code.
- When using a JavaScript filter, the code receives a set of messages and returns one or more messages.
- When used in a conditional connector, the code receives a message and returns a boolean value (True or False).

JavaScript Filter

The most common uses of JavaScript are in the JavaScript filter. The JavaScript filter is a flexible and powerful tool. It can be used to perform simple tasks such as assigning properties to a message, or for complex logic. The filter is found in the Filter Toolbox under Common filters. In this module, we will focus on message manipulation using the JavaScript filter.

JavaScript Basics

JavaScript is a powerful object-oriented scripting language which is interpreted at execution, as opposed to a compiled language which is translated to low level code prior to execution. When a Rhapsody component using JavaScript is checked in, the script is compiled to an intermediate byte-code which is evaluated each time the component processes a message.

JavaScript was primarily designed to provide advanced functionality for web pages, but it is a highly functional language able to be used in other environments. At a high level, the JavaScript environment implemented in Rhapsody supports:

- An object-oriented approach.
- Statements that can span multiple lines and are ended with a semicolon (;) - every statement must end with a semicolon.
- Case-sensitive statements (next.getField(path); is different to Next.GetField(Path);)
- Comments in one of two forms:
 - in-line comments prefixed by // (double slash)
 - comment blocks prefixed by /* (slash, asterisk) and terminated by */ (asterisk, slash)
- Nested instructions that are encapsulated in { and } (opening and closing curly brackets).
- Logic elements including support for branching and looping.
- Loosely typed variables (that is, the data type held by a variable is not constrained and is evaluated each time the variable is accessed).
- Strings are enclosed with single, double quotes, or backticks. Strings enclosed with backticks can span multiple lines.
- A large library of functions and methods.

The following example illustrates some of these concepts:

```
// Declare variables and set their value
```

```
const maxValue = 22;
```

```
let workingValue = 10;
```

```
// Update workingValue if greater than maxValue
```

```
if(workingValue > maxValue) {  
    workingValue = maxValue;  
}
```

Further reading

Rhapsody-specific JavaScript documentation is at [JavaScript Object Reference\(opens in a new tab\)](#). This includes reference documentation for the various objects provided by Rhapsody. These objects will be discussed later in this lesson.

A huge amount of information on JavaScript already exists on the Internet. It is not the intention of this course to duplicate that information. A good starting point for anyone wanting to know more about JavaScript can be found in the links below:

- [Mozilla: A re-introduction to JavaScript\(opens in a new tab\)](#)
- [Mozilla: JavaScript documentation\(opens in a new tab\)](#)
- [Wikipedia: JavaScript syntax\(opens in a new tab\)](#)

Be aware that many tutorials will focus exclusively on the use of JavaScript in a web browser, and will include features not applicable to Rhapsody.

Variables in JavaScript

In JavaScript, variables are used to contain data we are working on or referring to, such as a patient identifier or reference to a message. Before a variable can be used, it needs to be created. We can create variables in one of three ways, depending on its intended use.

const

When the data a variable refers to is not expected to change, use `const` to create a read-only (constant) variable. As `const` variables are read-only, code errors are raised when the data in a variable is accidentally changed.

let

When you need to create a variable that can be updated, use `let`.

var

Variables created with `var` are an older style of a `let` variable with more loosely defined [scope\(opens in a new tab\)](#) and declaration rules. This can result in unexpected behavior.

Variable best practices:

- You only need to declare the variable once.

- Use const variables whenever possible. This prevents errors caused by unintended changes to the data in a variable.
- Give variables meaningful names for maintainability and self-documentation.

Variable Types

Variables in JavaScript are loosely typed. You can declare a message by typing const, let, or var then the variable name. The data type of the variable is based on the data type of the value assigned to the variable.

For example, let's consider the following statement.

```
let one = '1';
```

This statement creates a variable called “one” and assigns it a string value of “1”. Simply having quotes around the 1 creates a variable with a data type of string. Values encapsulated in quotes (either double or single) are evaluated as strings (text), and those without quotes are evaluated as floating point numbers. There is no distinction between different numeric data types (that is, between integer and floating point numbers).

You can use either single or double quotes in your JavaScript. It is recommended to choose either single or double quotes and use that style consistently.

JavaScript variable data types are dynamic. You could assign a string value to a variable, then an integer to the same variable later in the code, or even an array.

Variables, Existence, and the Null Value

As JavaScript variables are not typed, conversions will occur implicitly if required. A variable may have one of the following types:

Variable value	Description
null	The null object has a type of null and a value of null.
"" (that is, the empty string)	The variable is set to an empty string either by direct assignment or as the result of an operation.
Any value	The result of an assignment
undefined	The variable exists but has no value set. This may occur by either of the following mechanisms: - The variable is declared but has no value set. For example: <code>var tmpValue;</code> Note that undefined is the default state for a variable. - You reference an index in an array that does not exist. For example:

```
const descriptions = ['Reception', 'Surgery Suite', 'Radiology', 'Laboratory', 'Community'];
```

```
if ( descriptions[siteIndex] == undefined ) {
    // siteIndex points to undefined description value
    next.setProperty('Site', 'Unknown');
} else {
    next.setProperty('Site', descriptions[siteIndex]);
}
```

Testing Variable values

JavaScript supports the standard set of comparison operators which return a boolean value. Note that it supports the additional notion of strict equality, which means the variables may be compared on both value and type. This is required because of the implicit type conversions carried out by JavaScript and because an uninitialized variable has the value undefined, not null.

Comparison operator	Description
==	is equal to
!=	is not equal to
===	is exactly equal to (type and value)
!==	is not exactly equal to (type and value)
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

Be careful when testing values to ensure that the correct condition is being tested. In particular, when testing for null values, the strict equality operator (===) should be used.

Values should be tested using the if statement which has the form

```
if ( expression ) {
    // Perform actions
}
```

The expression may take the form of an equality test using the operators listed above. Actions in the block may be any valid JavaScript statements.

A variable may also be tested with the form if (variable) with the following results:

Variable result	Behavior
variable is undefined	false
variable is null	false
variable is a number	false - if the value is 0 (zero) or the special case NaN (not a number, for example the result of division by zero). true - all other cases.
variable is a string	false - if the value is the empty string (" or ""). true - all other cases.
variable is an object	true

Looping and Nesting

Statements may be grouped together in a block using { and } (opening and closing curly brace). Statement blocks are normally used to provide conditional execution (that is, they are executed only if a specific condition occurs) or to iterate (use the same code a number of times).

Conditional execution involves the evaluation of a test condition which returns true or false. **JavaScript** supports the if ... else structure to manage conditional execution, and additionally allows multiple conditions. For example:

Simple Condition

```
if(hour < 17) {  
    // Execute work hours code  
  
} else {  
    // Execute after hours code  
  
}
```

Complex Condition

```
if(hour < 8) {  
    // Execute after hours code  
  
} else if(hour < 17) {  
    // Execute work hours code  
  
} else {  
    // Execute after hours code  
  
}
```

Code blocks may also be nested if desired (for example, to refine conditions within a condition block).

Looping

JavaScript supports looping based on the value of a condition, as well as iterating. In both cases, take care to ensure that loop structures terminate correctly. Creating an infinite loop, one which does not terminate, will have dramatic effects on CPU usage.

A conditional loop executes while a condition is true, and is constructed using the while statement; for example:

```
while(hour == 1) {  
  
    // Execute this code block  
  
    // Ensure that the value of hour is updated in the loop  
  
}
```

An iterating loop uses the for construct to loop through the code block a number of times. For example:

```
for(var i = 0; i < names.length; i++) {  
  
    // Loop through an array  
  
}
```

In the example above, note the following:

- The loop variable may be declared in the loop specification, and may be re-used elsewhere.
- The plus plus operator (++) increments the value of the loop variable by one for each pass over the loop.
- The value of the loop variable should not be altered in the code block.
- Arrays are zero-based, so their first element is addressed at index 0. The last item in the array is one less than the length of the array.

The for ... in construct allows iteration through the properties of an object; for example:

```
const person={fname:"John", lname:"Doe", age:25};
```

```
for(x in person)
{
    // Iterate through the properties of person

    txt = txt + person[x];
}
```

Nesting code blocks

Code blocks may be nested within the preceding level, provided the nested block is completely encapsulated in the preceding level. That is, the opening and closing braces of the nested level must be within the opening and closing braces of the preceding level, and must not overlap the opening and closing braces of another block at the same level.

The Rhapsody JavaScript Editor

The JavaScript editor in Rhapsody contains a number of features that make editing JavaScript easier. Most of these features will be familiar to anyone who has used an advanced text editor such as Notepad++.

When a new JavaScript filter is created, it contains some commented out JavaScript statements that illustrate common tasks performed using the filter. These include:

- Simple date handling
- Getting and setting the body of the message as a string (ignoring the encoding)
- Handling iterations in a collection, including updating the message content and setting a property
- Calling shared JavaScript libraries
- Getting and setting the message body as JSON

As you configure the filter, delete any unused default code.

Syntax Highlighting

The Rhapsody JavaScript editor uses a variety of colors to identify different types of code content. When reviewing or editing some JavaScript, these colors can be used to aid in understanding of the code.

Multiple line indent

As code is added to a JavaScript block, you may need to reformat sections of code to maintain good formatting. To change the indentation on multiple lines of text, select all lines then press the Tab key to indent the text to the right or press Shift + Tab to move the block of text to the left.

Code Tips

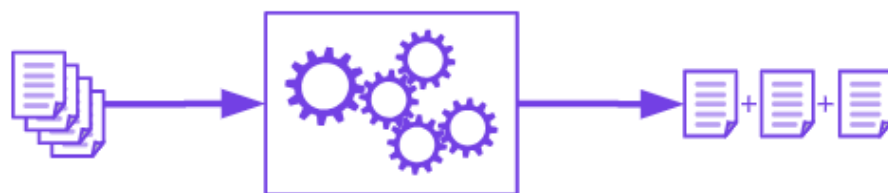
Hovering your mouse over either a methods or variable in your code will display a popup that provides details on the selected code element. Where desired, you can use JSDoc comments to add descriptions to your code that will appear in the code tips.

Code Completion

As you type into the JavaScript editor, a window similar to the tip window will appear containing suggestions based on what is being typed. At anytime you can accept the highlighted suggestion by pressing Enter. The suggestions can be further refined by continuing to type, or by using the ↑ and ↓ keys to select the required keyword from the list. The Enter or Tab key is used to select the required keyword.

The Input and Output Messages

Every filter in Rhapsody accepts an input collection containing one or more messages, performs some work on this collection, and generates an output of one or more messages.



Filter processing overview

For all the filters we have encountered in Rhapsody so far, the set of input and output messages is managed for us by the filter. In the JavaScript filter, we have access to all input and output messages as we configure the filter.

Input and Output in the JavaScript filter

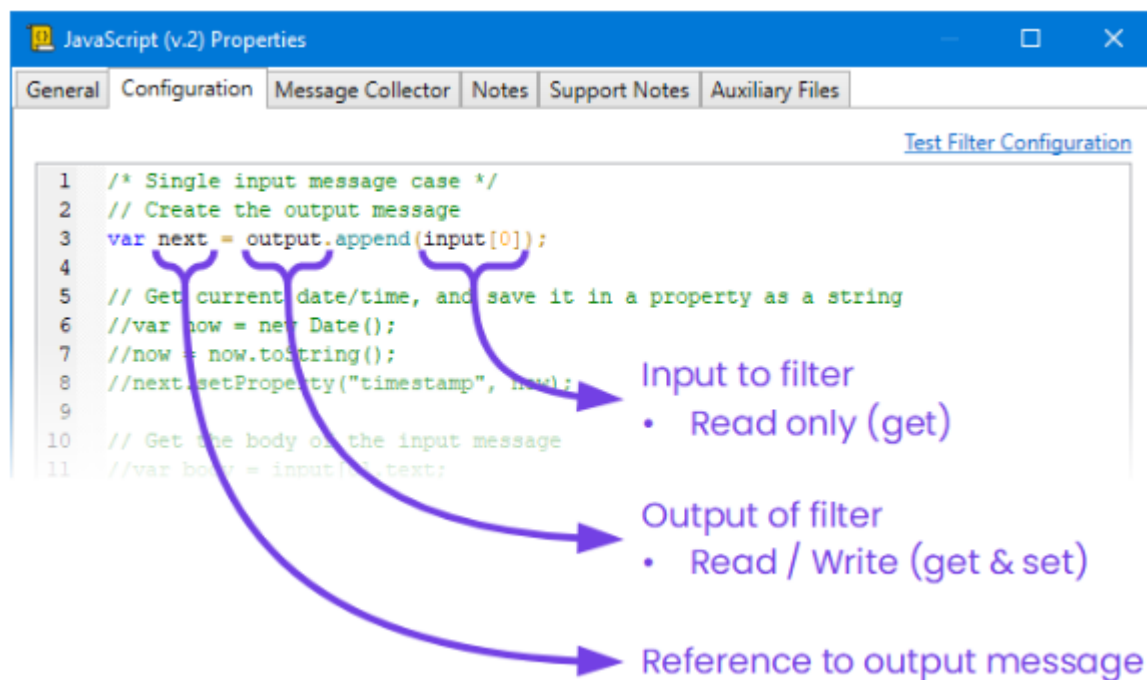
In the JavaScript filter, the collection of input messages is represented by the input array. Each item in the array represents an individual message within the input collection, with the first message accessed as `input[0]`. The input array is read-only within the JavaScript filter, you can get details for each message, but the input messages cannot be updated.

The group of output messages generated by the filter is accessed with the output array. Output messages are created by appending individual input messages to the output array. The append operation copies a message and all its properties to the output array. Additional output messages can be created by repeatedly appending to the output array. Output message are read/write, so can be modified once created.

The first line of code in the default configuration for the JavaScript filter creates an output message from the first input message and returns a reference to the new output message in the next variable.

```
var next = output.append(input[0]);
```

For JavaScript filters where a Message Collector has not been configured, this will normally be the first line of code in your filter configuration.



By convention, when working with a single message in the JavaScript filter, we assign the output message to a variable called next. All changes we wish to make to the message are made via this variable.

Managing the Input and Output Arrays

The Input and Output arrays are defined as a [MessageCollection\(opens in a new tab\)](#) object. The length property references the number of messages in the array. Access an individual message in the array by specifying its index within the array, such as input[0].

When working with the Output array, use the following methods to add or remove messages:

- `append(template)` adds a new message to the collection using an existing message as a template. This copies the properties and body of the template message to the new message. The template message is normally an input

message. Repeatedly calling this method adds additional messages to the filter output.

- `remove(index)` removes the specified message from the collection. The index is zero-based.

• Examples

- When working with a collection of input messages, the following code creates an output message for each input in the set:

```
for (let i = 0; i < input.length; i++) {  
    const next = output.append(input[i]);  
  
    // Add size of input collection to each output message  
    next.setProperty('InputCollectionSize', input.length);  
}
```

If a different output message is required for each destination system, the `append()` method can be used multiple times. In this example, we have not used the usual convention of using `next` to refer to the output message. This lets us quickly identify each output in the code.

```
// Create pathology output message  
const pathologySystem = output.append(input[0]);  
pathologySystem.setProperty('Destination', 'Pathology');  
  
// Make other changes to output message here  
  
// Create radiology output message  
const radiologySystem = output.append(input[0]);  
radiologySystem.setProperty('Destination', 'Radiology');  
  
// Make other changes to output message here
```

The Input and Message

The Input message is a read-only object (referred to as a `ROMessage` in the [Rhapsody Documentation](#)[\(opens in a new tab\)](#)). We can only get details for an input message. The methods all have the form `get...()`.

The message consists of:

- The body
The content of the message. This may be treated as:
 - A binary object, accessed either by referencing the property body or calling the method `getBody()`. The message content is handled as an array of bytes. This may cause issues if the content is encoded as multibyte text.
 - Encoded text, accessed by either the property text or the method `getText(encoding_name)`
- Properties, accessed by the method `getProperty(Property_name)`.

Accessing Message Content

When a message definition is associated with the route or filter, the message is parsed, and its content is accessible. Similarly, where the message is XML encoded, implicit XPath addressing is available if no message definition is specified.

Addressing Fields

Fields in an EDI message are addressable by specifying the path to the field using either a / (slash) or . (dot) notation. The path consists of the named elements from the root of the message to the required field. For example, the field path to the MessageType sub-field in the MSH segment of an HL7 v2.4 message is: MSH.MessageType.MessageType. When working with XML messages, the field is specified using an XPath expression.

The Input message only supports the ability to retrieve values from the message using the `getField()` method. This will return the field values at and below that location. For example:

- `getField('MSH')` returns the full MSH segment as a string.
- `getField('MSH.MessageType')` returns the values contained in the MSH.MessageType field (MessageType, TriggerEvent and MessageStructure sub-fields).
- `getField('MSH.MessageType.MessageType')` will return only the value of the specified field.

Repeating structures are identified by their index in the list, the index base being 0 (zero). For example, the first patient name in the PID.PatientName field is specified as `PID.PatientName[0]`. Note that you will need to be aware of the specific definition in use. **EDI Explorer** is helpful in visualizing and determining the message structure to a specific field.

The value returned by `getField()` is as follows:

- If the path is present in the message, the field value is returned as a string.
- If the path is valid, but has no value or the field does not exist in the message, null is returned.
- If the path is not valid, an exception will be returned. This may be caught using try / catch, or the message will be sent to the filter's Error Connector due to the processing error.

The number of repeats of a field is determined by calling `getRepeatCount(field_path)`. If the path does not exist in the message, this method returns zero.

Accessing Properties

Properties are name / value pairs associated with the message. The property name is defined when the property is created, and the value may contain any content. While there are no explicit limits on the length of a property value, properties should not hold large quantities of data.

Once created, the property is associated with the message until it has completed processing, and it may be modified at any processing step. Each change to a property is saved to the datastore.

Properties may be flagged for their values to be indexed as the message leaves the route. Such indexed properties are available for inclusion in search conditions for an output search, but their values are not available until persisted at the output to the route.

Properties are accessed by the following JavaScript methods:

- `getProperty('property_name')`: retrieves the value of the named property.
- `getPropertyAsList('property_name')`: retrieves a list of property values for the named property and requires that the property has been created as a list.
- `getPropertyNames()`: retrieves a list of the names of all the message properties.

The Output Message

The output message is a writable [message object \(opens in a new tab\)](#). It supports all the methods of the input message object `get...()` and extends support to a number of methods which write to the message `set...()`. This allows the message to be manipulated and updated as required.

Managing Message Fields

Field values in the output message are addressable provided the message is parsable. The following methods are supported:

- `getField(field_path)` retrieves the value of the field.

- `setField(field_path, value)` sets the value of the field. You may need to check for existence of the field in the message before setting its value.

Managing Message Properties

Property values may be retrieved and set on the Output message object using the same methods as the Input message:

- `getPropertyNames()` returns an iterator containing the names of all the message properties.
- `getProperty(property_name)` retrieves the value of the property.
- `getPropertyAsList(property_name)` retrieves the list of values for the specified property as a comma separated list.

Where the message has been output from a filter via the Error Connector, the message object also supports the `getErrors()` method, which returns an array of all parsing errors encountered during processing of the message.

The Output message additionally supports methods for setting property values:

- `setProperty(property_name, value)` updates or creates the named property.
- `addPropertyValue(property_name, value)` adds the value to the property list of that name, creating it if necessary.

The Output message also permits manipulation of Indexed properties, which are persisted on the output of the message from the route and are available to use in a search condition:

- `indexOutputProperty(property_name)` flags that property to be indexed.
- `removeIndexingForProperty(property_name)` removes the indexing.

Example

In the following example, a JavaScript filter has been configured to look at an HL7 ACK message and identify all the errors in the message that contain an error code in the range 200-299.

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
let numRejections = 0;
```

```
// Iterate over all errors in message and count all those in the range 200-299

for (let curError=0; curError < next.getRepeatCount('ERR.ErrorCodeAndLocation');
curError++){

    const errorCode = next.getField('ERR.ErrorCodeAndLocation[' + curError +
    '].CodeIdentifyingError.Identifier');

    // Check to see if the error code is in the range 200-299.
    if (errorCode >= 200 && errorCode <= 299) {

        numRejections++;

    }

}

// Set number of rejections as a message property
next.setProperty('NumberRejections', numRejections);
```

Notice the following aspects of the code:

- All code inside a set of curly brackets is indented to make the code easier to read.
- The `getRepeatCount()`, `getField()` and `setProperty()` functions are all performed against the output message, which is defined as `next` in the first line of code.
- The field passed to the `getField()` function has to include the index of the error being inspected. The index is dynamically added to the field by string concatenation.

Manipulating the Message

The body of the message is available in a number of forms and may be extracted and manipulated as required.

Body as Text

The message body may be assigned to a variable using the text property of the message object. For example:

```
let body = next.text;
```

The text property provides access to the encoded message body using the encoding set for the message. Note that setting the encoding causes the message to be translated according to that encoding's rule set – it does not provide a translation between encodings.

To get the message body with a specific encoding, use the `getText()` function. This may be required where the message encoding has been incorrectly set, or where the encoding can only be determined at runtime:

```
let body = next.getText('UTF8');
```

The variable created is a string and may be manipulated using JavaScript methods and functions. For example, an HL7 message might be decomposed into an array of segments by using the `split` function with the carriage return character as the split value, or portions of the body may be updated by sub-string manipulation.

Once manipulation is complete, the updated content can be written back to the message by setting the text property of the output message:

```
next.text = body;
```

The message body can be set with a specific encoding with the `setText()` function:

```
next.setText(body, 'UTF8');
```

Body as a Byte Array

The message body may be accessed as a byte array using the `body` property of the message. For example:

```
let body = next.body;
```

This allows the direct manipulation of the byte values. This requires considerable caution if the encoding is not an 8-bit encoding scheme.

Once manipulation is complete, set the `body` property of the output message:

```
next.body = body;
```

EDI Message

When the message being processed by the JavaScript filter has an accompanying Rhapsody Messaging Toolkit definition, we can directly access and edit fields in the message.

Using the following test message and the academy-HL7v2.4-ADT.s3d message definition:

```
MSH|^~\&|Rhapsody|Rhapsody|SimplePAS|Test|20221121105056||ADT^A03|JCA03235a|T|2.4
```

```
EVN||20221121105046
```

```
PID|||84568-4564^^^MRN||Cardinal^John||19491112|M
```

```
PV1||E||T0034-20221121|||||||||E541987
```

```
DG1|1||T14.1^Open Wound|||A
```

```
DG1|2||S51.0^Open wound of elbow|||F
```

The getField(), setField(), and getRepeatCount() methods can be used to edit the message as follows. This approach is sufficient for the majority of message editing that occurs in the JavaScript filter.

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Set message property from date of birth field in the message
```

```
next.setProperty('DateOfBirth', next.getField('PID.DateTimeOfBirth'));
```

```
// Update SendingApplication field
```

```
next.setField('MSH.SendingApplication.NamespaceID', 'HealthyOrg');
```

```
// Count all final diagnosis in message and set message property
```

```
const diagnosisCount = next.getRepeatCount('DG1')
```

```
let finalDiagnosis = 0;
```

```
for (let i=0; i < diagnosisCount; i++) {
```

```

    if (next.getField('DG1[' + i + '].DiagnosisType') == 'F') {
        finalDiagnosis++;
    }
}

```

```

next.setProperty('FinalDiagnosisCount', finalDiagnosis);

```

Where additional control is required, the underlying Rhapsody Messaging Toolkit API can be accessed using the `getReadableEDIMessage()` and `getWritableEdiMessage()` methods. Message elements are accessible through the `getElementAt(field_path)` method, and values may be updated using the `setValue(value)` method of the element.

For example, using the previous test message would edit it as follows:

```

// Create the output message

```

```

const next = output.append(input[0]);

```

```

// Get writable version of EDI message

```

```

const ediMessage = next.getWritableEdiMessage();

```

```

// Set message property from date of birth field in the message

```

```

next.setProperty('DateOfBirth',
ediMessage.getElementAt('PID.DateOfBirth').getEncodedValue());

```

```

// Update SendingApplication field

```

```

ediMessage.getElementAt('MSH.SendingApplication.NamespaceID').setValue('Healthy
Org');

```

```

// Count all final diagnosis in message and set message property

```

```

const dg1 = ediMessage.getElementAt('DG1');

```

```

const diagnosisCount = dg1.getElementCount();

```

```

let finalDiagnosis = 0;

for (let i=0; i < diagnosisCount; i++) {
  if (dg1.getElementAt(i).getElementAt('DiagnosisType') == 'F') {
    finalDiagnosis++;
  }
}

next.setProperty('FinalDiagnosisCount', finalDiagnosis);

// Delete preadmit number if visit number contains a value
const pv1 = ediMessage.getElementAt('PV1');

if(pv1.getElementAt('VisitNumber').isPresent()) {
  pv1.getElementAt('PreadmitNumber').setEmpty();
}

```

HL7 Messages using HAPI

HL7 messages can be processed using the [HAPI Library\(opens in a new tab\)](#) even when they do not have an associated definition. Full details of the HAPI API calls can be found at [HAPIMessage Object\(opens in a new tab\)](#).

Using the previous HL7 example message, we could edit it using the HAPI message object as follows:

```

// Create the output message

const next = output.append(input[0]);

// Create transformable hapi message

const hapiMessage = next.getHapiMessage();

```

```

// Set message property from date of birth field in the message (PID.7)
next.setProperty('DateOfBirth', hapiMessage.getValue('PID', 0, 7, 0, 1, 1));

// Update SendingApplication field (MSH.3.1)
hapiMessage.setValue('MSH', 0, 3, 0, 1, 1, 'HealthyOrg');

// Count all final diagnosis in message and set message property
const diagnosisCount = hapiMessage.numberOfSegmentRepeats('DG1');
let finalDiagnosis = 0;

for (let i=0; i < diagnosisCount; i++) {
  if (hapiMessage.getValue('DG1', i, 6, 0, 1, 1) == 'F') {
    finalDiagnosis++;
  }
}

next.setProperty('FinalDiagnosisCount', finalDiagnosis);

// Delete PID Segment
hapiMessage.deleteSegment('PID');

// Update output message
next.text = hapiMessage.getOutput();

```

XML Messages

When a message has either an XSD definition or no definition associated with it, Rhapsody will attempt to parse the message as XML when using the getField(), setField(), and getRepeatCount() methods. When processing the message as XML, XPath expressions are used to identify fields.

The xml property on the Output message can be used to update the output message to contain the string assigned to it. If the updated message is not valid XML, an exception will be thrown and the message sent to the Error Output of the filter.

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Create a new XML Message (use backticks so that the string can span multiple lines)
```

```
next.xml = `<patient>
```

```
  <name>
```

```
    <given>Patrick</given>
```

```
    <family>Stewart</family>
```

```
  </name>
```

```
  <gender>Male</gender>
```

```
</patient>`;
```

```
// Set message property from a field in the message
```

```
next.setProperty('Gender', next.getField('/patient/gender'));
```

```
// Update given name in message
```

```
next.setField('/patient/name/given', 'Pat');
```

```
// Add a new node to the XML
```

```
next.setField('/patient/name/type', 'Legal');
```

JSON Messages

Rhapsody supports the standard JavaScript `JSON.parse()` and `JSON.stringify()` methods for working with JSON (JavaScript Object Notation) messages.

If the following JSON message was received by Rhapsody:

```
{  
  "patient": {
```

```
"identifier": "AQ-59016-FH",  
"name": {  
  "firstName": "Bruce",  
  "middleName": null,  
  "familyName": "Wayne"  
},  
"DateOfBirth": "1939-04-03",  
"Gender": "Male"  
}  
}
```

The following example would extract the patient identifier and add a new field to the message:

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Parse JSON from text
```

```
const data = JSON.parse(next.text);
```

```
// Add identifier as message property
```

```
next.setProperty('PatientId', data?.patient?.identifier);
```

```
// Add a new field into the message
```

```
data.patient.validated = true;
```

```
// Update JSON in output message
```

```
next.text = JSON.stringify(data);
```

When selecting fields in a JSON message, Optional Chaining (adding a ?. between each item in the path) can be used to prevent missing or optional items from causing the selector to fail with an error.

JavaScript Counters

JavaScript counters are set and used through the `incCounter()`, `getCounter()`, and `setCounter()` functions, and can be used to allocate unique (integer) identifiers. These functions can be used to guarantee that the identifier is unique, for example from the `InputTime` property, which under certain circumstances may not be unique. If a message set does not have a unique identifier, then one can be generated by using the `incCounter()` function in a JavaScript filter.

In this example, we set the `MSH.MessageControlID` field to a unique value for each message that uses the JavaScript filter.

In this example, we set the `MSH.MessageControlID` field to a unique value for each message that uses the JavaScript filter.

```
// Create the output message
```

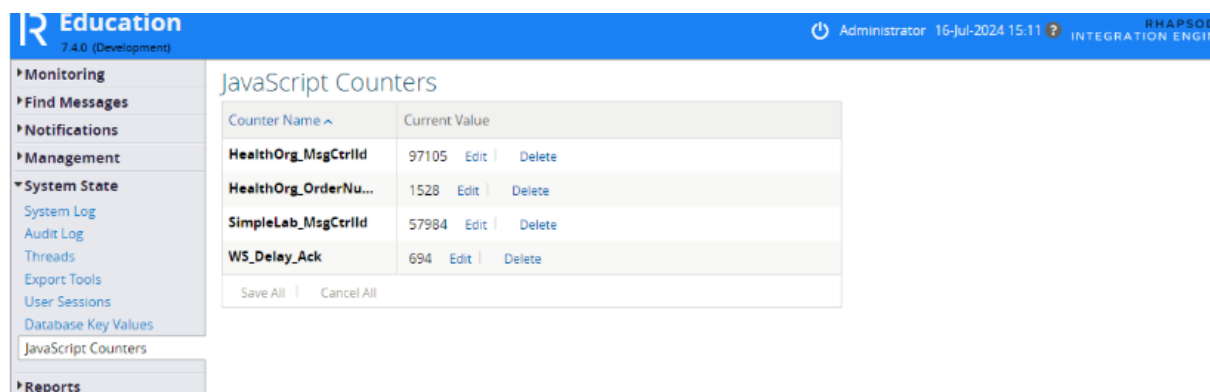
```
const next = output.append(input[0]);
```

```
// Set the message control id with a unique value
```

```
next.setField('MSH.MessageControlID', incCounter('HealthOrg_MsgCtrlId', 1));
```

Managing Counter Values

The **JavaScript Counters** page in the Management Console is used to view, edit, and delete message counters.



The screenshot shows the 'JavaScript Counters' page in the Management Console. The interface includes a sidebar with navigation options: Monitoring, Find Messages, Notifications, Management, System State (with sub-items: System Log, Audit Log, Threads, Export Tools, User Sessions, Database Key Values), and Reports. The main content area displays a table of counters with columns for Counter Name and Current Value. The table lists four counters: HealthOrg_MsgCtrlId (97105), HealthOrg_OrderNu... (1528), SimpleLab_MsgCtrlId (57984), and WS_Delay_Ack (694). Each row has 'Edit' and 'Delete' links. At the bottom of the table are 'Save All' and 'Cancel All' buttons. The top of the page shows the 'Education' logo, version '7.4.0 (Development)', and user information 'Administrator 16-Jul-2024 15:11'.

Counter Name	Current Value
HealthOrg_MsgCtrlId	97105
HealthOrg_OrderNu...	1528
SimpleLab_MsgCtrlId	57984
WS_Delay_Ack	694

If a counter is deleted from the Management Console, the next call to the `incCounter()` function will recreate the counter and set it to a value of zero.

Functions

You will normally only use the `incCounter()` function when working with counters in Rhapsody.

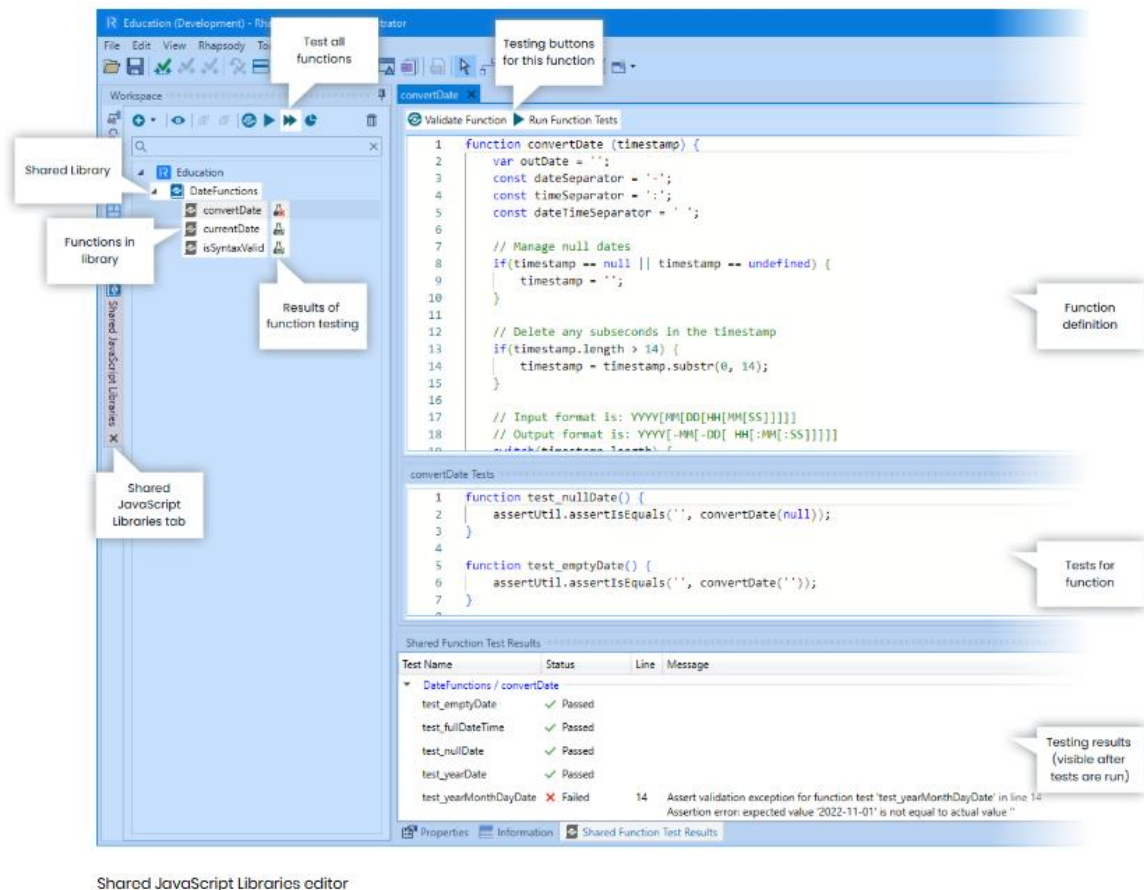
Function	Description
<code>incCounter(counterName[, increment])</code>	Increments the global counter with the specified name by the value provided and returns the new value. This is the safest way to use counters as it will atomically retrieve the current value, increment it, set the new value, and return it. The increment parameter defaults to
<code>getCounter(counterName)</code>	Retrieves the current value of the global counter with the specified name. The value is always returned as an integer, and the counter is created with an initial value of zero if it does not currently exist.
<code>setCounter(counterName, int)</code>	Sets the global counter with the specified name to the value provided. If no counter with the given name exists, one will be created.

JavaScript Shared Functions

Within Rhapsody, it is possible to create JavaScript functions that can be shared between any JavaScript component in your solution. These are useful where the same snippets of JavaScript are required in multiple places within the Rhapsody configuration. Shared Functions are grouped into libraries. This lets you group related functions together. The maximum size of a shared function is 16 Kilobytes.

Creating and editing Shared Functions

The shared functions editor is accessed by selecting either the **View** menu or the **Managers** drop-down, then **Shared JavaScript Libraries**. This enables the **Shared JavaScript Libraries** tab in the workspace, which displays all your shared functions.



New libraries and functions are created from the workspace. Right-clicking on either a library or function presents an extended set of options, including the **Show Uses** option that lists all the places where a given shared function or library has been used. When editing an existing function, this option should be used to help ensure changes will not break your existing configuration.

Double-clicking on a function opens it for editing. Once editing and testing of the function is complete, you will need to check it in before it can be used in your configuration.

Function Testing

During the development of a Shared JavaScript Function, you will need to test the function to make sure it is correct. This is done with two tools:

Validate Function checks the syntax of the function to make sure it is valid. This will find errors such as invalid JavaScript expressions and missing brackets in your code.

Run Function Tests runs the contents of the test functions. One or more test functions can be created for each shared function. These will normally be configured to call your shared function, then run one or more assertions against the result to check that the

function behaves as intended. When you first create a new shared function, code comments in the test will list all available assertions.

If you want to run function tests for all of your functions at the same time, use the icon at the top of the workspace.

A summary of the test appears to the right of the function name in the workspace. Details of the test appear in the **Test Results** panel that appears at the bottom of the screen once the test has been run.

Using A Shared Function

To use a shared JavaScript function, the required library of functions must be included in the code for your filter. This is done by including the `require()` function to reference the desired library. The function returns a link to the library that is used to reference functions within the shared library.

For example, to access a library called `DateFunctions` the following code would be used:

```
// Require the DateFunction library of functions
```

```
const dateFns = require('DateFunctions');
```

```
// Create the output message
```

```
const next = output.append(input[0]);
```

```
// Get current datetime for insert into message
```

```
const newMsgDate = dateFns.currentDate();
```

```
...
```

Best Practices

The goal when using JavaScript in your configuration should be to make it efficient, effective, logical and supportable. The best practices below will help you achieve this goal.

Formatting

As with code that you create in any other programming language, proper formatting of JavaScript code will make the intention of the code more clear and self-documenting.

Consistency

Be consistent in your coding style. If you use single instead of double quotes to surround strings, be consistent throughout, don't mix quotes in the same code. Use the same variable naming style throughout, whether it's camel-case, Hungarian or lower/upper case. Consistency is key to clean, efficient code.

Indentation

You should properly indent logic statements (if, for, while, switch) to clearly delineate the statement block from surrounding code.

```
if (messageType == 'arrive') {  
    // Set event date as now  
    const now = new Date();  
    eventDate = now.toString();  
} else {  
    // Get event time from message  
    eventDate = next.getField('/attendance/event/startDate');  
}  
  
// Loop through all the input messages  
for (let i = 0; i < input.length; i++) {  
    // Add the message from the input to the output body  
    body += input[i].text + newline;  
}
```

Comments

Comments in JavaScript perform two important functions:

Documentation

You should add comments to make the function of your JavaScript clear. Your comments should also explain any code that is not immediately clear. Your comments should also contain the creation date, the author and any change history made to the code.

```
// We are using an input collation that guarantees the order that messages arrive in the filter.
```

// The original query message will always be first, with the ADR response second.

```
const qry = input[0];
```

// Create the output message

```
const adr = output.append(input[1]);
```

// Copy content from qry to ADR message.

```
adr.setField('QRD.QueryFormatCode', qry.getField('QRD.QueryFormatCode'));
```

```
adr.setField('QRD.QueryID', qry.getField('QRD.QueryID'));
```

// Copy select message properties to ADR message

```
adr.setProperty('Siteld', qry.getProperty('Siteld'));
```

Deactivating executable code

When developing, testing, or troubleshooting a filter, it is often useful to comment out some code to stop it from executing. Always leave an explanation as to why the code is commented out. Including your name or initials and the date helps document when the change was made and provide follow-up contact details.

```
/* Commented out while debugging error processing in OpenPAS - BD 2020-06-19
```

```
// Generate ACK
```

```
if (ackCode == 'AA') {
```

```
    next.setField('MSA/TextMessage', 'Success');
```

```
} else {
```

```
    next.setField('MSA/TextMessage', next.getProperty('ErrorCondition'));
```

```
}
```

```
*/
```

If the code is no longer needed, delete the code. This includes the code samples included from when the JavaScript filter was created.

Testing

When developing your JavaScript filters, filter testing should be used to test your code. This helps you identify faults in your code as it is written and improves the confidence you have in your code. The test messages used with filter testing should include both expected and unexpected messages. Also consider how different message properties will be interpreted by the JavaScript code in your filter.

Multiple JavaScript Filters

Keep the number of JavaScript filters to a minimum by combining code. Whenever a filter alters the message body, it is written to the datastore. Having too many filters can cause unnecessary writes to the datastore and increase its size on disk.

If there are multiple JavaScript filters in-line on one route, you may want to consider combining the code to one JavaScript filter if possible. An exception is if a JavaScript filter is performing multiple functions and has hundreds of lines of code, making it difficult to understand. In this case, you may split it into multiple JavaScript filters, or use JavaScript functions to simplify the code.

Logging

Only use `log.debug()` when using Rhapsody logging objects for development and troubleshooting JavaScript code, because the Management Console's default log level for writing to the log files is INFO. If `log.info()` is used for debugging or troubleshooting, every time the JavaScript code executes at runtime, it will write to the Management Console and log files. This could cause unnecessary clutter in the logs, making it difficult to troubleshoot real issues.

Shared libraries

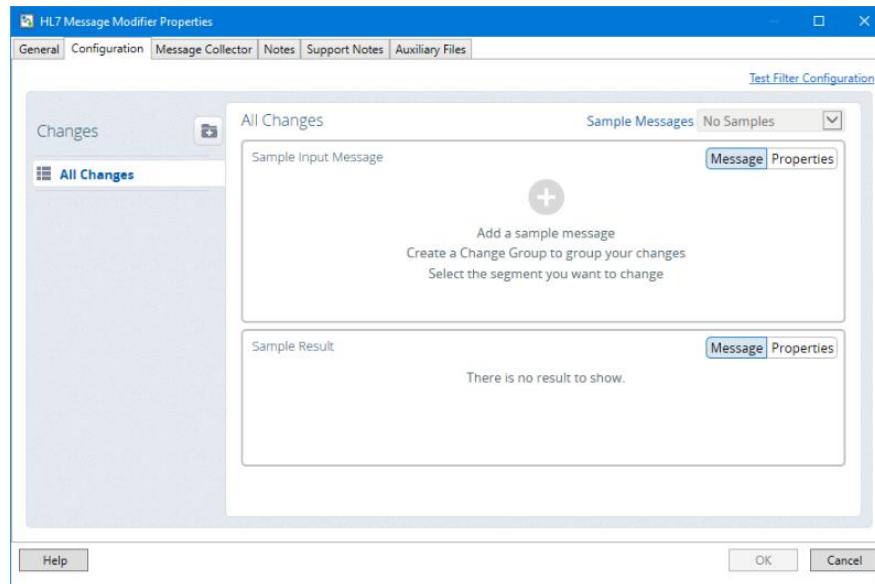
If you have code that will be re-used, consider creating a shared JavaScript function. This will make it easier to manage the code if any updates or changes need to be made.

Message Modifier

Overview

The **HL7 Message Modifier** is a Rhapsody filter that operates on HL7 messages using an inbuilt generic message definition. It is designed to quickly and easily perform most commonly-used modifications on HL7 messages being processed by Rhapsody. It is the best choice for manipulating messages when only small changes are required.

When the Message Modifier filter is first created, the Sample Input Messages panel includes text that outlines the filter's configuration steps.



The following pages in this module will walk you through the process of configuring the Message Modifier.

System Requirement

The HL7 Message Modifier uses an embedded HTML viewer to display the filter configuration within the Rhapsody IDE. This requires that Internet Explorer Enhanced Protection Mode is disabled on the computer running the Rhapsody IDE.

The first time a HL7 Message Modifier filter is opened, you will be prompted to install the Rhapsody Management Console certificate so that you can configure the filter. If you have insecure (HTTP) access to the Management Console configured, you will be able to bypass the certificate setup requirement.

Add a Sample Message

Configuration of the message modifier works by performing all modifications in real time against a set of one or more sample messages. The first task is to add some sample messages to the filter. The sample messages should be carefully chosen to cover all likely input scenarios.

Select the Sample Message link at the top right corner of the configuration tab. This will display the Sample Message Manager window that allows you to upload one or more sample messages.

Sample Message Manager

Load and edit sample messages to test your modifications

Sample Message Name ^

There are no sample messages.

New Sample Message

Filename

Message Name *

+ Add as a New Message

Close

Select **Browse** to select a HL7 message from the local file system, then **Upload** to display the contents of the sample message. Provide a **Message Name** for the message that describes the message. The body of the message can be edited if required before finally selecting the **Add as a New Message** link to add the file to the filter.

Once all sample messages have been added, select **Close** to return to the main configuration screen.

Editing a Sample Message

An existing sample can be edited or deleted from the same screen by selecting the name of the message at the top of the window.

Sample Message Manager

Load and edit sample messages to test your modifications

Sample Message Name ^	
Electrolytes	✕
Full Blood Count	✕

New Sample Message

Filename

Message Name *

```

MSH|^~\&|QS-LIS|Rhapsody^City Hospital^HL7|CDR|Test|20090922154122||ORU^I
PID||84568-4341^^MRN^MRN-104341R^^MRN^NHR||HEADROOM^Max^P||1968012302^
ORC|OK|||||Qx10-000824^Martin^J^^Dr.^^^QS^^^tst
OBR|1|647827|^CANNIS|638991^CBC|||20071127201500|||20071127201500||Qx
OBX|1|NM|Hb^Hb||16.2|g/L|11.1-15.5|H||F|||||20071127201500
OBX|2|NM|HCT^HCT||34.4|L|33-46||F|||||20071127201500
OBX|3|NM|MCV^MCV||87.0|fL|80-100||F|||||20071127201500
OBX|4|NM|MCH^MCH||34.2|pg|27.0-35.0||F|||||20071127201500
OBX|5|NM|MCHC^MCHC||33.1|g/L|32.0-36.0||F|||||20071127201500
  
```

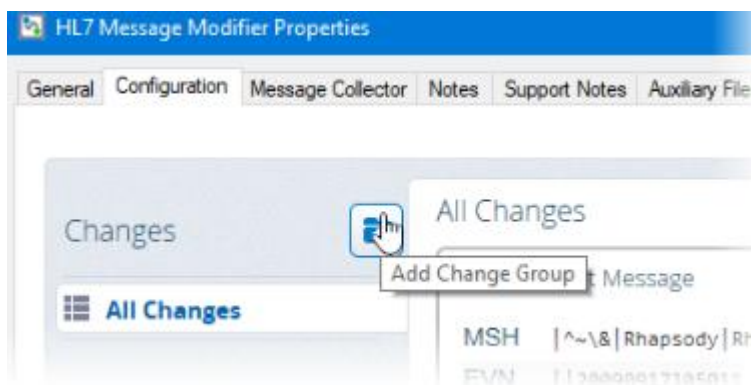
Once you have finished reviewing or editing the sample messages, select Close to return to the main configuration screen.

Create a Change Group

All mapping rules in the Message Modifier are grouped together. This allows related changes to be grouped together.

To create a change group, select the **Add Change Group** icon above the **All Changes** list.

This will display a screen for the group. Update the **Group Name** field with a suitable name for the group. The **Description** field should be populated with details of the change along with the reason for it.



Select the Segment You Want to Change

To select a segment to modify, first make sure the appropriate change group has been chosen by selecting the name of the desired change group. Then either select the **Define a Segment to Modify** link, or select the appropriate segment in either the **Sample Input Message** or **Sample Result** message.

This will display a screen that displays just the segment to be modified. From this screen, it is possible to add rules that change the segment or add message properties to the outbound message.

It is possible to add additional segments to a change group by selecting the change group, then following the steps above and selecting the subsequent segments.

Add Actions and Rules

The current sample message appears in the **Sample Message** panel. The results of all actions currently defined for a segment will be showing in the **Result** panel. Changes will be highlighted with a green background for easy identification.

The **Properties** button displays the properties that have been created or updated against the Result.

Result		Message	Properties
Message Properties			
Property Name		Value	
PatientCity		Auckland	

Actions

The **Action** button is used to add an action to the current segment which will cause something to change or be updated. An action can also be added by selecting the field to be modified in the **Sample Segment**.

An action can do one of four things:

- change the value of the field using one of the in-built functions
- change value of a message property
- clear the value of the field
- delete the segment

Conditions

Selecting **Condition** will add a condition rule, allowing actions to be selectively run.

Once a condition has been added, it is possible to

add **Otherwise** and **Otherwise If** rules to the condition to allow actions to be selectively run if the main condition is not met.

The screenshot displays a rule configuration interface with a light gray background. It contains several horizontal bars, each representing a condition or action. Each bar starts with a three-dot menu icon on the left. The first bar is yellow and contains the text 'If OBX.11 Equals X'. Below it is a white bar with 'Delete Segment' and 'OBX'. The third bar is yellow and contains 'Otherwise If OBX.11 Not Equals 0'. Below it are two white bars: 'Change Field OBX.15.1 To Value Good Lab' and 'Change Field OBX.15.2 To Value GoodLab'. The sixth bar is yellow and contains 'Otherwise'. Below it are two white bars: 'Clear Field OBX.5' and 'Clear Field OBX.6'.

⋮ If	OBX.11	Equals	X
⋮ Delete Segment	OBX		
⋮ Otherwise If	OBX.11	Not Equals	0
⋮ Change Field	OBX.15.1	To Value	Good Lab
⋮ Change Field	OBX.15.2	To Value	GoodLab
⋮ Otherwise			
⋮ Clear Field	OBX.5		
⋮ Clear Field	OBX.6		

Moving Actions and Conditions

It is possible to change the order that actions are run and what actions are associated with a condition by dragging each action or condition. To do this, place the mouse over

the dots at the left of an item so that the cursor turns into a four-way arrow. Click and drag the item to the indented location.

The indentation of an item indicates that it is run as part of the condition immediately above it.

Animated example

This example illustrates adding an action to delete an OBX segment when the OBX.11 (observation result status) field contains a value of X (results not cannot be obtained for this observation), otherwise OBX.19 (date time of analysis) is updated with the current date time.

Testing Changes

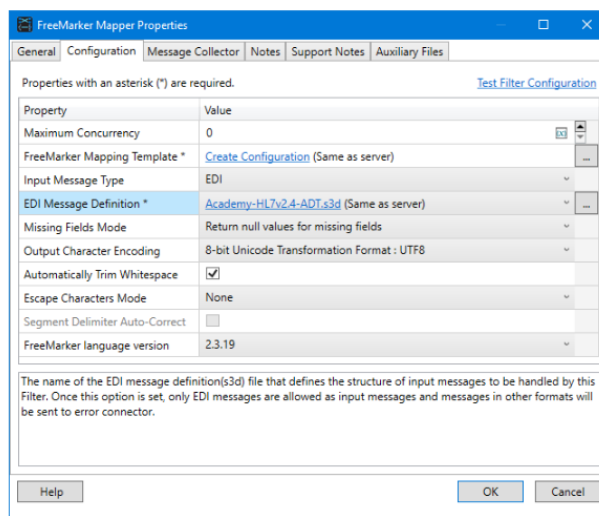
Once the various changes have been made to the message, select the **All Changes** link in the top left corner of the screen to view all changes that have been performed. Select each of the sample messages from the drop-down list in the top right corner of the screen and review the **Sample Result** to confirm that the expected result is being generated.

FreeMarker Mapper

FreeMarker Mapper

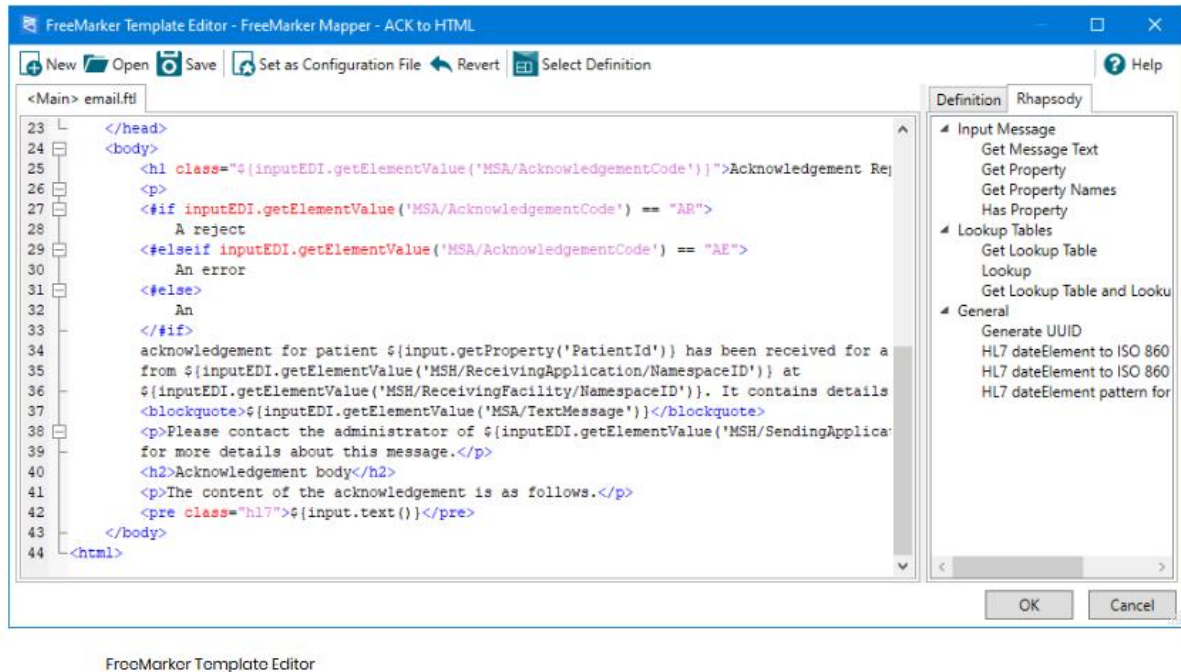
The **FreeMarker Mapper** filter uses the FreeMarker template language to convert input messages formatted as either EDI and XML into an arbitrary text message. Normally this will be an XML formatted message; however it could just as easily be an HTML or free text email message. The format of the output text message is determined by a template that can include fields and properties from the input message.

The **Configuration** tab of the FreeMarker filter's **Properties** dialog is shown below:



FreeMarker Properties dialog

Once a message definition has been loaded into the **EDI Message Definition** property, the remainder of this filter's configuration is completed by selecting the **Create Configuration** link in the **FreeMarker Mapping Template** value field. The **FreeMarker Template Editor**, is shown below:



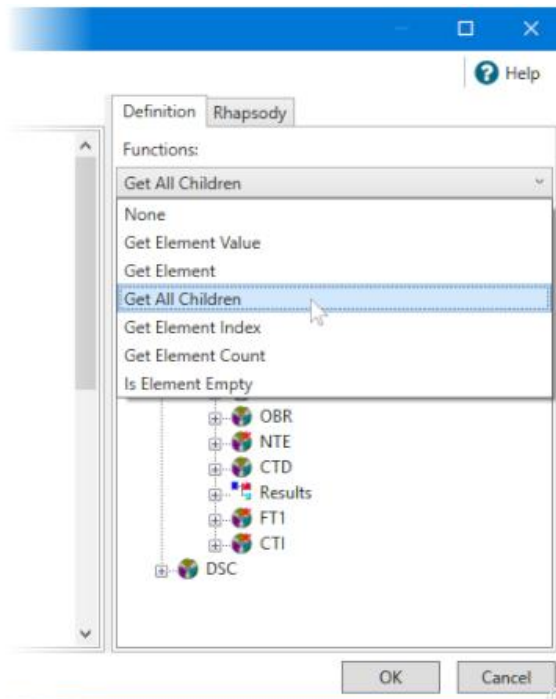
The FreeMarker Mapper filter supports multiple versions of the FreeMarker language. Some changes between language versions are not backward compatible. Details of the version changes can be found [here \(opens in a new tab\)](#). The version used by the filter can be selected using the **FreeMarker language version** property. By default, this property is set for compatibility with older Rhapsody configurations (version 2.3.19).

FreeMarker Template Interface

The icons at the top of the editor are used to **Add**, **Open** and **Save** a FreeMarker Template File (.ftl). If a FreeMarker configuration contains more than one file, the **Set as Configuration File** button makes the currently selected file the main file in the FreeMarker configuration. All files associated with the filter are accessible from the filter's **Auxiliary Files** tab.

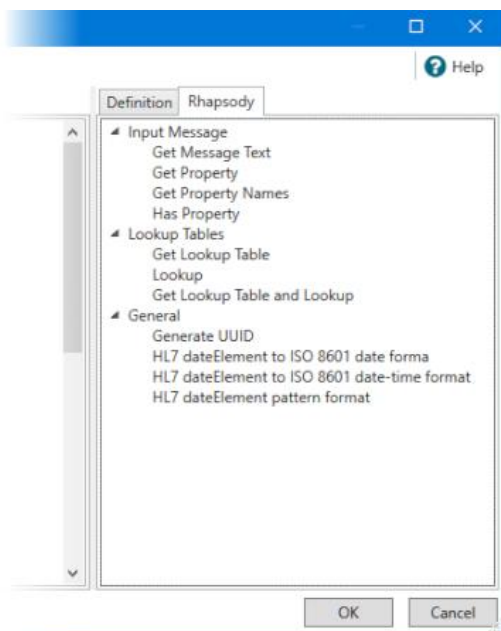
A message definition can be associated with the FreeMarker filter either on the **Configuration** tab of its **Properties** dialog, or by selecting **Select Definition** in the template editor. Once a definition has been loaded, message types included with it can be seen in the **Definition** tab on the right of the editor screen.

Individual message types can be expanded and their associated fields dragged into the **Editing** panel for inclusion in the mapping code. Optionally, selecting a function then dragging a message field into the editor window will add the appropriate FreeMarker statement into the configuration.



FreeMarker Template Editor - Definition Functions

Next to the Definition tab is the **Rhapsody** tab, which contains links to several functions that can also be dragged into the template.



FreeMarker Template Editor - Rhapsody Functions

Reference Documentation

The following reference documents provide information on the construction of template files within Rhapsody.

- Rhapsody FreeMarker API Reference in the Rhapsody Documentation: [FreeMarker API Reference\(opens in a new tab\)](#)
- FreeMarker Template Author's Guide: [http://freemarker.org/docs/dgui.html\(opens in a new tab\)](http://freemarker.org/docs/dgui.html(opens in a new tab))

FreeMarker Template Examples

The following sample scripts illustrate the configuration of the **FreeMarker Mapper** filter.

Generate XML from a HL7 message

This example takes an HL7 message and generates an XML message based acknowledgement message.

```
<ack>
```

```
<sendingSystem>${inputEDI.getElementValue('MSH.SendingApplication.NamespaceID')}</sendingSystem>
```

```
<id>${inputEDI.getElementValue('MSH.MessageControlID')}</id>
```

```
</ack>
```

Generate an HTML report from an HL7 acknowledgement

In this example, we extract information from an HL7 acknowledgement message to generate an HTML report. The report also contains information gathered from a PatientId message property. An if statement in the template alters the text of the report based on the type of acknowledgement received.

```
<html>
```

```
<head>
```

```
<title>Acknowledgement Report</title>
```

```
<meta http-equiv="content-type" content="text/html; charset=utf-8">
```

```
<style type='text/css'>
```

```
body{
```

```
font-family: sans-serif;
```

```
}
```

```
.hl7{
```

```
padding: 3px;
border: 1px solid;
font-family: courier new,courier,monospace;
white-space: pre;
overflow: auto;
}
```

```
.AE, .AR {
    background-color: #facabc;
}
```

```
</style>
```

```
</head>
```

```
</head>
```

```
<body>
```

```
<h1
class="{inputEDI.getElementValue('MSA/AcknowledgementCode')}">Acknowledgeme
nt Report</h1>
```

```
<p>
```

```
<#if inputEDI.getElementValue('MSA/AcknowledgementCode') == "AR">
```

```
    A reject
```

```
<#elseif inputEDI.getElementValue('MSA/AcknowledgementCode') == "AE">
```

```
    An error
```

```
<#else>
```

```
    An
```

```
</#if>
```

```
    acknowledgement for patient ${input.getProperty('PatientId')} has been received for
a message
```

```
    from ${inputEDI.getElementValue('MSH/ReceivingApplication/NamespaceID')} at
```

`${inputEDI.getElementValue('MSH/ReceivingFacility/NamespaceID')}`. It contains details of:

`<blockquote>${inputEDI.getElementValue('MSA/TextMessage')}`

`<p>Please contact the administrator of
${inputEDI.getElementValue('MSH/SendingApplication/NamespaceID')}`

`for more details about this message.</p>`

`<h2>Acknowledgement body</h2>`

`<p>The content of the acknowledgement is as follows.</p>`

`<pre class="hl7">${input.text()}</pre>`

`</body>`

`</html>`

Viewing output

The following report was generated using the above script in response to an HL7 acknowledgement message with an acknowledgement code of AR.

Acknowledgement Report

A reject acknowledgement for patient JFC9825 has been received for a message from LabSoft v8.3 at City Hospital. It contains details of:

Invalid Patient Id

Please contact the administrator of MetaStore 0.9a for more details about this message.

Acknowledgement body

The content of the acknowledgement is as follows.

```
MSH|^~\&|MetaStore 0.9a|City Hospital|LabSoft v8.3|City Hosp  
MSA|AR|3214|Invalid Patient Id
```

Generate a FHIR Resource from an HL7 message

The following example builds an XML formatted FHIR Patient resource from an HL7 ADT message. This example use a list iterator to add all patient identifiers to the generated FHIR message.

```

<?xml version="1.0" encoding="UTF-8"?>

<Patient xmlns="http://hl7.org/fhir">

  <id value="pat1"/>

  <text>

    <status value="generated"/>

    <div xmlns="http://www.w3.org/1999/xhtml">

      <p>Patient ${inputEDl.getElementValue('PID.PatientName[0].GivenName')}
      ${inputEDl.getElementValue('PID.PatientName[0].MiddleInitialOrName')}
      ${inputEDl.getElementValue('PID.PatientName[0].FamilyName.Surname')}, MRN =
      ${inputEDl.getElementValue('PID.PatientIdentifierList[0].IDNumber')}</p>

    </div>

  </text>

  <#list inputEDl.getElementAt('PID.PatientIdentifierList').allChildren as identifier>

    <identifier>

      <use value="usual"/>

      <type>

        <coding>

          <system value="http://hl7.org/fhir/v2/0203"/>

          <code value="${identifier.getElementValue('IdentifierTypeCode')}" />

        </coding>

      </type>

      <system
value="${identifier.getElementValue('AssigningAuthority.NamespaceID')}" />

      <value value="${identifier.getElementValue('IDNumber')}" />

    </identifier>

  </#list>

  <active value="true"/>

```

```

<name>

  <use value="official"/>

  <family
value="\${inputEDI.getElementValue('PID.PatientName[0].FamilyName.Surname')}" />

    <given value="\${inputEDI.getElementValue('PID.PatientName[0].GivenName')}" />

    <given
value="\${inputEDI.getElementValue('PID.PatientName[0].MiddleInitialOrName')}" />

  </name>

  <gender value="\${LookupTables.get('FHIR_Gender').lookup('v2',
inputEDI.getElementAt('PID.Sex'), 'FHIR')}" />

  <birthDate
value="\${HL7v2DateTimeFormatter.date(inputEDI.getElementAt('PID.DateTimeOfBirth'))
}">

    <extension url="http://hl7.org/fhir/StructureDefinition/patient-birthTime">

      <valueDateTime
value="\${HL7v2DateTimeFormatter.dateTime(inputEDI.getElementAt('PID.DateTimeOfBi
rth'), false)}" />

    </extension>

  </birthDate>

</Patient>

```

It would be possible to create a similar FreeMarker template to generate a JSON formatted FHIR resource.